

Quantitative Design of the Gaffers Clipboard Rating Algorithm

Executive Summary

The quantification of a football performance into a single number from 0.0 to 10.0 has long been a complex and controversial process. At its core, it is an exercise in dimensionality reduction - distilling a web of complex and often interconnected statistics down to a single number. Industry-standard algorithms, such as those employed by platforms like *Sofascore* and *Fotmob*, have made significant strides in this area. However, fans frequently view them critically, as these systems carry inherent biases and operate as 'black boxes' with little transparency into how they actually weigh and analyse data.

The Gaffer's Clipboard Analytics Engine seeks to address these flaws by providing a transparent, mathematically rigorous 0.0 to 10.0 scale. While it has its own limitations (relying on statistics extracted directly from EA FC/FIFA screens), the engine operates completely offline. It feeds match telemetry through a strict pipeline involving data standardisation, principal component analysis, and contextual heuristics. By evaluating player performance solely on what unfolds on the virtual pitch, the engine offers users a clear, unbiased view of performance, fostering trust and confidence in tactical decisions and long-term tracking.

I. Introduction and Philosophy

In today's modern world, there is an increasing emphasis on viewing things through the lens of data and statistics. Whether assessing a company's performance over a financial quarter, a student's academic abilities in school, or the supposed success of a government, organisations and individuals increasingly seek the ability to strip away human error and bias to view performance through an objective lens. Moreover, the field of sports is no different. Whilst there are a few other similar fields, like politics, few others come close to the level of tribalism and, therefore, inherent biases found in sports and amongst their stakeholders. Along with how prevalent sports can be in our daily lives and the growing economic value they create, this explains why sports teams, fans, and others involved are increasingly focused on quantifying sports into objective data.

However, this is harder than may be initially thought, and harder with some sports than others. For example, analysing the performance of a sprinter is easy; there is only one real metric that matters - their time. Moreover, if one wants to go deeper, with cameras and basic timing technology, it is relatively easy to track their top sprint speed, their acceleration over the first x meters, and so on. Team sports, on the other hand, are a different ball game, and football more than most. In the NBA, for example, it is somewhat easy to determine who did well in a game without watching it, simply by looking at simple metrics: points, assists, rebounds, steals, turnovers, etc. However, whilst football may have similar metrics: goals, assists, passes, tackles, shots, etc, these are nowhere near as informative. Whilst in basketball, fans may debate about who is the best ever by looking at career averages like points or field goals, the only statistic in football that comes somewhat close to that level of prominence in these debates is goals scored, and that is only one aspect of a player, and an even smaller aspect if that player is not a striker.

Part of why football is so much harder to quantify is that luck is a more significant factor in it than in other team sports. With such long matches and yet such small scorelines, it is common to see players and teams perform well, yet draw with or lose to teams supposedly performing much worse, simply because they had one moment of luck. Another is the sport's complexity and the expectations placed on players. Whilst a golfer may only be expected to put the ball towards the hole, a single footballer may be expected to consistently pass the ball to his teammates successfully (but if he is making safe passes too much that may be seen as bad), tackle

opposing players and intercept their passes, cover a lot of ground across the pitch, create chances for their teammates, and get in good positions to shoot themselves. This is before one looks at aspects of their performance that are even harder to quantify in a single number. Take pressing: the action of a player or team applying pressure to the opponent whilst they have the ball, attempting to force turnovers, minimise time on the ball, and win possession. It is incredibly hard to scale how well a player or team performs at this into a single number.

Therefore, reducing all aspects of a player's performance, a matrix of spatial, temporal and event-based telemetry, into one unified player rating poses significant mathematical and epistemological challenges. The engine evaluates a variety of data types expressed in different ways - some unbounded, others not normally distributed - and attempting to quantify them on a finite scale.

At least amongst fans, a few apps and their commercial algorithms dominate this space (like *Sofascore*, *Fotmob* and *Whoscored*). These are increasingly significant in shaping how fans view players, influencing public perception and even professional scouting. Fans who have not watched a game will still often confidently form an opinion on how a player played based on a single number. However, at the same time, a lot of fans are noticing the serious deficiencies and drawbacks of these models, from their strong attacking bias (teams of the week and of tournaments generated by these services often do not represent a realistic team, usually packed too full of attacking players), to their lack of transparency (serious frustration comes from the black box nature of these algorithms, fans not being allowed to understand the inputs or how the algorithm works, only the output). The so-called "eye test" still is, and always will be, important in evaluating a performance, and fans often express annoyance when mathematical ratings do not reflect what they have watched with their own eyes.

Performing the previously mentioned analyses, not on real football matches but on simulated ones via a video game, is even more complicated. EA FC's simulation and the stats it generates from it have their own drawbacks. Whilst the video game has its own match rating system built in, it is deeply flawed. The in-game algorithm evaluates players in a tactical vacuum, heavily over-indexing on raw goal involvements while routinely ignoring positional responsibilities. For example, a fullback explicitly instructed to stay back and defend will consistently receive artificially low match ratings for failing to register attacking output. Furthermore, the game's temporal scaling is highly volatile; an 85th-minute substitute can drastically inflate their match score with a mere handful of successful passes, breaking managerial immersion. These limitations are exacerbated by how the Gaffer's Clipboard application works - to make it easier to use, and the data logging process smoother and simpler, only 16 stats are collected directly from each player's performance, along with 16 stats collected on each team's collective performance in the match. Additionally, the goal of developing a useful, lightweight desktop application for users on all kinds of computers discourages the use of heavy machine learning libraries or massive cloud servers. Instead, the aim was to extract heavier aspects of the algorithm into a purely development-only phase, storing their outputs with the program, and then, when analysing a user's own data generated as they use the app on their own computer, simply using pure maths, lightweight numpy arrays and contextual heuristics. By decoupling heavy data science from the live scoring engine, the application delivers enterprise-grade analytical depth locally and instantaneously, without any external dependencies.

Before the engine can process the match data, it must confront the strict limitations of this native telemetry API. Real-world football analytics engines (such as those powered by *Opta*) benefit from hundreds of granular micro-events, including "Key Passes," "Interceptions," and "Progressive Carries." The Analytics Engine, however, is constrained by the rigid, 16-metric outfield matrix provided by the game's post-match extraction.

Because the engine lacks advanced spatial data, it cannot measure a player's creativity purely through counting stats. To bridge this gap, the algorithm synthesises a 17th metric - Expected Threat (xT) - derived from a combination of a player's passing efficiency, physical intensity, and ball-retention profile. This final 17-element array serves as the absolute boundary of the engine's worldview. Everything the algorithm

understands about a player's performance must be mathematically deduced from this highly restricted telemetry matrix.

Deep Dive: The Outfield Telemetry Matrix >

01a - The Outfield Telemetry Matrix

1. The Native API Constraints

The Analytics Engine operates on a strictly bounded data diet. The native extraction tool captures only 16 base metrics for outfield players. These metrics are heavily biased toward traditional counting stats and lack the granular transition data (e.g., interceptions, progressive carries, key passes, aerial duels) standard in modern data science.

To standardise the evaluation of playing time, all volume-based counting stats are normalised "Per-90" (p90) rate before entering the matrix.

2. The 16-Element Native Array

The raw telemetry matrix is partitioned into four distinct tactical categories:

Attacking & Output:

1. goals_p90
2. assists_p90
3. shots_p90
4. shot_accuracy (%)

Possession & Distribution:

5. passes_p90
6. pass_accuracy (%)
7. dribbles_p90
8. dribble_success_rate (%)

Defensive Actions:

9. tackles_p90
10. tackle_success_rate (%)
11. possession_won_p90
12. possession_lost_p90

Physicality & Discipline:

13. distance_covered_p90
14. distance_sprinted_p90
15. fouls_committed_p90
16. offsides_p90

Note on shots: The native telemetry records total shots. Before the 17-element vector enters the scoring pipeline, shots are decomposed into non_goal_shots_p90 — shots minus goals, clipped to zero. This isolates pure chance creation from goal-scoring output, so a striker who scored twice from two shots is not rewarded twice for the same events.

3. The 17th Dimension: Synthetic Expected Threat (xT)

Because the native 16-element array completely lacks spatial progression metrics (e.g., the difference between a safe sideways pass and a line-breaking through ball), the engine generates a 17th synthetic feature: **Expected Threat** (`xt_bonus_p90`).

This derived metric acts as a mathematical proxy for ball progression and spatial dominance. The engine constructs it from three components: the player's sprint ratio (the proportion of total distance covered at high intensity, capturing physical intent and line-breaking runs), their passing volume and accuracy (capturing technical execution and distribution quality), and a possession control ratio (which filters out defensive recovery sprints from genuine attacking build-up play, by comparing possession won against possession lost).

The positional scalar applied to the formula also varies by role — attackers and attacking midfielders receive a higher base multiplier than defensive midfielders and centre-backs, reflecting the different degree to which each position is expected to generate progressive threat. Full mathematical derivation and calibration details are in [03a - Expected Threat Synthesis](#).

This finalised 17-element array (T_{17}) serves as the sole input vector for the downstream PCA eigen-decomposition and positional heuristic pipelines.

Despite the existing limitations, the goal of this algorithm was to eliminate the black box commonly used by industry-standard services. The architecture takes messy OCR-extracted telemetry data and employs a transparent, purely mathematical approach to distil it into a single player rating. Similar to professional alternatives, there was a strong emphasis on context - considering positional variance, intelligent tactical roles, team performance, and match configurations. Ultimately, this algorithm aims to synthesise these complex contextual variables into a single, transparent metric, transforming raw telemetry into a definitive and unbiased reflection of true on-pitch value.

II. Data Pre-Processing & Normalisation

Every statistical model, regardless of its complexity, is bound by one fundamental law: the quality of its output is entirely dependent on the quality of its inputs. After validating the extracted telemetry, the application verifies that the values fall within expected ranges before routing them through the algorithm and applies common-sense rules, such as goals cannot be greater than shots, and assists cannot be greater than passes. However, in its rawest form, this extracted telemetry is simply a collection of isolated event counts, inherently devoid of context. Feeding this unadjusted data directly into a complex scoring algorithm would create highly suspect outputs, as it treats all events as equal, regardless of the opportunities provided to the player.

In football, this concept of opportunity is inextricably linked to time. Evaluating a player purely on their gross accumulation of stats heavily skews the data in favour of those who complete a full match, whilst punishing those who are substituted on or off. These raw counts hide efficiency; a player registering 15 passes in 90 minutes has performed quite poorly in that area, whilst a player who has achieved the same number in a 15-minute cameo has performed quite well. To accurately judge a performance, the data must reflect a rate of action rather than a raw sum. Across football statistics, this is usually expressed as a rate per 90: the number of actions the player would have completed had he played 90 minutes. Statistics shown across a season, such as goals, are often presented this way: goals per 90. This method shows how well a player has performed relative to the opportunities they have had, accounting for injuries and adding parity to player comparisons.

When translating this logic to a video game simulation like EA FC, the temporal problem doubles. Unlike real-world football, simulation telemetry is fixed to two clocks: the 90 minutes of the football match and a customisable half-length game setting. This setting sets the number of real-world minutes each half takes, which can range from 4 to 20 minutes, and the in-game 45 minutes scales accordingly. The problem this poses is that, with more real-world time to play, and for instance press the pass button more times, the volume counts will be higher in longer half-length matches. To ensure mathematical fairness and cross-match comparability,

the Analytics Engine must first pass all raw OCR-extracted data through a strict pre-processing pipeline. This phase standardises the volume metrics and masks low-volume anomalies, ensuring the foundational data is perfectly level before any scoring or positional weighting is applied.

Temporal Standardisation

To address the half-length problem, the engine introduces a baseline half-length setting that scales all cumulative volume stats, regardless of their original half-length. This baseline is set at 10 minutes, and all cumulative volume stats (passes, shots, tackles, etc.) are scaled to it. The maths to achieve this is simple, multiplying the raw stat by the ratio of the baseline half-length to the real half-length ($X_{adj} = X_{raw} \times \left(\frac{H_{base}}{H_{match}}\right)$). For example, if a user is playing 4-minute halves, then a midfielder completing 20 passes is playing quite well. However, if this were directly compared to a 10-minute half (where midfielders playing the full match usually average around 40 passes) without standardisation, it would look quite poor. To scale this volume up, the algorithm passes the metric through the standardisation formula ($20 \times \left(\frac{10}{4}\right) = 50$), and it is converted to its 10-minute half equivalent: 50 passes. This performance can now be fairly compared against a performance from a match with 10-minute halves.

Handling Substitute Volatility (Bayesian Smoothing)

While temporal standardisation fixes EA FC's custom clock, the engine must still apply the standard fix: per-90 normalisation. However, simply converting these adjusted stats into a per-90 rate introduces a new mathematical hazard: substitute volatility. If an 88th-minute substitute comes on and takes a single shot on target in two minutes of game time, this standard formula would give a per-90 rate of an absurd 45 shots per game. If the program fed that inflated rate directly into the scoring matrix, it would break the algorithm and generate a 10.0 rating for a two-minute cameo.

To prevent this, the engine demands proof of sustainability. It applies a statistical technique known as Bayesian smoothing, which acts as a mathematical anchor. By blending the player's observed stats with a historical positional average, the engine ensures that a player's rating is closely tied to the mean (generating a standard 6.0 rating) when they first step onto the pitch. The fewer minutes a player has on the field, the stronger this anchor holds, meaning their calculated output is largely dictated by historical expectations rather than their brief cameo. To break away from this baseline and earn an elite (or abysmal) rating, the player must physically accrue enough minutes, and consistent statistical output, to overpower the positional average.

Crucially, the anchor does not hold with equal strength across every statistic. It is calibrated to how quickly each type of event reveals a reliable rate. Statistics that accumulate continuously throughout a match, such as passes and distance covered, stabilise relatively quickly; even a 20-minute cameo generates enough evidence for a reasonable estimate. For less frequent events, tackles and dribbles, the anchor holds longer, requiring more time on the pitch before the observed rate is trusted. For the rarest, most match-defining events, goals, assists, and shots, the anchor is strongest of all, ensuring that a five-minute substitute who fires in a single shot is never projected as a 45-shots-per-game phenomenon. Full technical calibration of the tiered anchor system is detailed in the deep dive below.

Deep Dive: Bayesian Smoothing Architecture >

02b - Bayesian Smoothing Architecture

1. The Mathematical Hazard of Small Samples

As outlined in the theoretical methodology, extrapolating small-sample simulation data (such as 1 shot in 2 minutes) into a "per 90" rate relies on a linear assumption that fundamentally breaks at such extremes. The danger of this is not just the inflated number itself, but how it interacts with downstream calculations. When an unanchored, mathematically absurd extrapolation — such as 45.0 shots per 90 — enters a standard deviation (Z-score) matrix, it registers as a multi-sigma outlier. This artificially

warps the player's dimensional weight and completely bypasses positional constraints, mathematically guaranteeing a maximum match rating of 10.0 for a brief cameo with very little impact on the actual game.

2. The Metric-Calibrated Shrinkage Anchor

To resolve this, the engine utilises Bayesian smoothing: a statistical shrinkage technique that pulls low-confidence observations (those with small sample sizes) toward a high-confidence prior (historical averages).

The conceptual blending formula is as follows:

$$r_{smoothed} = \frac{M}{M + d} \cdot r_{obs} + \frac{d}{M + d} \cdot r_{prior}$$

- M : Minutes actually played.
- r_{obs} : The observed statistical rate.
- d (**Dummy Anchor**): A metric-specific constant representing the weight of historical expectation, calibrated to each metric's stabilisation rate (see three-tier system below).
- r_{prior} : The historical expected rate for that specific position.

The constant d dictates the critical "pivot point" — the exact minute mark where the weight of a player's live observed performance equals the weight of the historical expectation. Rather than applying a single universal anchor to every stat, the engine assigns d based on how quickly each metric's true rate stabilises. This produces three tiers:

- **High-Frequency Metrics** ($d = 15.0$)

Passes, distance covered, distance sprinted, possession won, possession lost.

These statistics accumulate continuously. After 15 minutes of football, a player has interacted with the ball and covered ground often enough that their observed rate is already a reasonable estimate. The pivot point falls at exactly 15 minutes — equal weight to live data and prior from the first quarter-hour. A full 90-minute match yields a live-data weight of $\frac{90}{90+15} \approx 85.7\%$, making the prior a marginal residual for complete matches.

- **Medium-Frequency Metrics** ($d = 30.0$)

Dribbles, tackles, fouls committed, offsides.

These occur regularly but not continuously. A 10-minute cameo may contain only one or two tackles, offering little statistical confidence. The pivot point falls at 30 minutes — the classic "impact sub" window, considered the minimum for meaningful tactical integration. A full match yields a live-data weight of $\frac{90}{90+30} = 75\%$.

- **Rare Events** ($d = 45.0$)

Goals, assists, shots.

These are discrete, high-stakes events that remain genuinely volatile even over a full match. A striker who scores twice in the first 10 minutes of their cameo should not be projected as an 18-goals-per-game phenomenon. The pivot point falls at 45 minutes, so only players who have seen substantial game time begin to earn real trust in their goal-threat rate. A full match still retains $\frac{90}{90+45} \approx 66.7\%$ live-data weight, acknowledging that even complete matches carry meaningful match-to-match variance for these events.

3. The Logic of Priors (r_{prior})

For Bayesian smoothing to function logically, the algorithm cannot use a blanket prior for all statistics. The engine bifurcates the prior based on the fundamental nature of the telemetry being evaluated.

Rare, Discrete Events ($r_{prior} = 0.0$)

Metrics such as goals, assists, and shots represent highly valuable, discrete, and inherently rare events. For these `rare_cols`, the prior must be strictly set to 0.0. If the algorithm used a historical average — for example, if a striker historically averages 0.6 goals per game — the blending formula would falsely inject fractional "ambient" goals into a substitute's baseline rate simply for stepping onto the pitch. In a mathematically rigorous engine, decisive match-altering actions cannot be assumed; they must be earned entirely from scratch through live telemetry.

Continuous, Volume Events ($r_{prior} = \mu_{historical}$)

Conversely, cumulative metrics such as passes, tackles, and distance covered represent continuous tactical execution. Even when a player is not actively on the ball, they are occupying space, pressing, and moving within a system. For these `volume_cols`, the prior dynamically queries the offline `means_stdts` dictionary to fetch the exact historical mean (μ) for the active positional role.

Using a 0.0 prior for these continuous events would incorrectly assume an 80th-minute defensive midfielder actively refused to pass or tackle, artificially dragging their efficiency scores toward zero. By anchoring volume metrics to the historical positional mean, the engine gives the player the "tactical benefit of the doubt" — assuming they are fulfilling the baseline responsibilities of their role at an average rate, until their live telemetry proves they are over- or under-performing.

4. Algorithmic Implementation (`_apply_bayesian_smoothing`)

Within the `MatchRatingsService`, this mathematical foundation is executed with a focus on application stability, efficient memory allocation, and defensive programming. The engine relies entirely on native Python dictionaries, avoiding heavy data-science libraries.

The anchor values are stored as a class-level immutable constant, mapping each metric to its calibrated tier:

```
DUMMY_WEIGHTS: Final = MappingProxyType(
    {
        # High-Frequency (Fast stabilization)
        "passes": 15.0,
        "distance_covered": 15.0,
        "distance_sprinted": 15.0,
        "possession_won": 15.0,
        "possession_lost": 15.0,
        # Medium-Frequency (Moderate stabilization)
        "dribbles": 30.0,
        "tackles": 30.0,
        "fouls_committed": 30.0,
        "offsides": 30.0,
        # Low-Frequency / Rare Events (Slow stabilization, high volatility)
        "goals": 45.0,
        "assists": 45.0,
        "shots": 45.0,
    }
)
```

The engine receives a dictionary of `normalized_metrics` (stats already adjusted for half-length) and routes them through a bifurcated pipeline.

The Rare Events Matrix

For discrete events (`rare_cols` : goals, assists, shots), the prior is 0.0 and the anchor is $d = 45.0$. Because the prior is zero, the right side of the blending formula is neutralised entirely. The engine evaluates these using a rapid dictionary comprehension, fetching d from `DUMMY_WEIGHTS` :

$$X_{p90} = \left(\frac{X}{M + d} \right) \times 90.0$$

The heavy $d = 45.0$ anchor suppresses extreme extrapolations from short appearances without requiring any positional database queries.

The Volume Matrix and Log-Mean Back-Conversion

For continuous tactical events (`volume_cols` : passes, tackles, distance, etc.), the engine fetches the historical mean (μ) for the player's position and applies the appropriate per-metric d :

```
d = self.DUMMY_WEIGHTS.get(col, self.DEFAULT_DUMMY)

col_stats = self.means_stds.get(pos, {}).get(
    f"{col}_p90",
    {"mean": 0.0, "std": 1.0}
)
league_average_p90: float = col_stats.get("mean", 0.0)
```

A critical detail applies to four volume statistics: `possession_won`, `possession_lost`, `fouls_committed`, and `offsides`. These are log-transformed before their means are computed in the offline calibration notebooks (as part of the log-normalisation pipeline described in Section III). This means the value stored in `means_stds` for these columns is $\mu(\ln(x + 1))$ — a mean in log-space — rather than a raw per-90 rate. Before it can serve as a Bayesian prior, it must be back-converted to the original statistical space:

```
if col in {"possession_won", "possession_lost", "fouls_committed", "offsides"}:
    league_average_p90 = float(np.expm1(league_average_p90))
```

`np.expm1(x)` computes $e^x - 1$, exactly reversing the $\ln(x + 1)$ transformation. Without this step, the prior would systematically underestimate these rates, dragging smoothed values toward an artificially compressed baseline.

Once the prior is correctly retrieved, the `dummy_stat` is computed — the literal anchor weight dropped onto the scales:

$$\text{dummy_stat} = r_{\text{prior}} \times \left(\frac{d}{90} \right)$$

$$X_{p90} = \left[\frac{X + \text{dummy_stat}}{M + d} \right] \times 90$$

Absolute Stability and Divide-by-Zero Safety

By executing the algebraically reduced formula above, the engine never divides by M . Because the denominator is strictly $(M + d)$ and every d is at least 15.0, the denominator can never equal zero — regardless of telemetry edge cases such as a player logged with 0 minutes played.

To prove that the optimised Python implementation mathematically matches the conceptual Bayesian formula, we map the variables as follows: X = raw volume, M = minutes played, d = metric-specific anchor (15.0, 30.0, or 45.0).

Step 1: The Conceptual Formula

$$r_{smoothed} = \left(\frac{M}{M+d} \right) r_{obs} + \left(\frac{d}{M+d} \right) r_{prior}$$

Step 2: Substitute r_{obs} for its raw mathematical components

The observed per-90 rate (r_{obs}) is calculated as $\frac{X}{M} \times 90$. Substituting this in:

$$r_{smoothed} = \left(\frac{M}{M+d} \right) \left(\frac{X}{M} \times 90 \right) + \left(\frac{d}{M+d} \right) r_{prior}$$

Step 3: Cross-Cancel and Align Denominators

In the first term, the M in the numerator and the M in the denominator cancel each other out:

$$r_{smoothed} = \frac{X \times 90}{M+d} + \frac{d \times r_{prior}}{M+d}$$

Because both terms share the same denominator ($M+d$), they combine into a single fraction:

$$r_{smoothed} = \frac{90X + d \cdot r_{prior}}{M+d}$$

Step 4: Align with the Python Implementation

The Python code structures the formula to multiply by 90 at the very end to format the output as a per-90 rate:

$$X_{p90} = \left[\frac{X + r_{prior} \times \left(\frac{d}{90} \right)}{M+d} \right] \times 90$$

Distributing the $\times 90$ into the numerator:

$$X_{p90} = \frac{90X + (r_{prior} \times \frac{d}{90} \times 90)}{M+d}$$

The 90 and 90 on the right side of the numerator cancel, leaving:

$$X_{p90} = \frac{90X + d \cdot r_{prior}}{M+d}$$

Conclusion: The formulas are algebraically identical. The proof holds regardless of whether d is 15.0, 30.0, or 45.0 — the reduced formula is a universal representation of the conceptual Bayesian blend for any metric-specific anchor value. The X_{p90} reduction safely calculates the exact same shrinkage rate without ever dividing by M .

III. Live Feature Engineering & Standardisation

With the raw telemetry successfully cleaned, standardised, and insulated against substitute volatility, the foundational data entering the algorithm is mathematically stable. However, even after this pre-processing, these stats cannot be fed directly into a rating matrix. In their current form, they lack two critical mathematical properties: dimension and equivalence.

Football is an inherently multidimensional sport, and evaluating a player requires balancing their offensive output, defensive contributions, and possession retention. Raw counting stats offer no way to compare these differing actions; one cannot simply add "3 tackles" to "50 passes" to deduce a match rating. Is 50 a good number of passes? And if it is, is it unlikely 50 is a similarly good number of tackles, or shots, for example.

To resolve this problem of comparing apples to oranges, the Analytics Engine executes a rigorous live feature engineering pipeline. This phase synthesises an advanced metric using a mathematical proxy to replace missing spatial context. It then compresses certain skewed statistics into a comparable distribution before converting all telemetry into a universally comparable scale, and applies confidence-weighted volume scaling to ensure that a player's calculated efficiency is fully earned.

The Expected Threat (xT) Proxy

In real-world football analytics, the Expected Threat (xT) metric, like related metrics such as Expected Goals (xG), is calculated using precise X/Y pitch coordinates. Expected Threat uses these to measure how much a player's actions increase the team's probability of scoring - how much threat they generate. It does this by mapping a player moving the ball from a low-value zone (such as around the halfway line) to a high-value zone (such as in or around the opponent's penalty box). However, the data extracted from EA FC post-match stats screens entirely lacks spatial context. In the game's raw dataset, a five-yard sideways pass between two centre-backs is indistinguishable from a 40-yard line-breaking pass from a playmaker. This discrepancy is made worse by the fact that no similar metric, such as Expected Assists (xA), is collected, which would show the quality of the chances created, regardless of whether the playmaker's teammates finished them. Without an xT metric, the algorithm would inherently reward passive, stat-padding possession just as highly as dynamic, attacking football.

To bridge this spatial void, the Analytics Engine synthesises an xT proxy by combining available metrics to approximate Expected Threat. This heuristic is grounded in real-world sports science - research from the CIES Football Observatory and tracking data from UEFA Euro 2016 demonstrate that while total distance covered has little correlation to match success, high-intensity sprinting is statistically linked to positive goal differentials and is most commonly seen among attacking players.^{[1][2]} Furthermore, industry analysis shows that elite teams combine this physical intensity with high passing volume to generate a true attacking threat.^[3] By mathematically binding a player's sprint ratio to their passing efficiency, the engine accurately isolates box-to-box midfielders, overlapping fullbacks, and direct wingers who drive progressive, line-breaking possession.

However, sprinting hard does not automatically mean generating attacking threat. A centre-back conceding a high press may cover a high proportion of their distance at sprint speed without ever advancing the ball. To filter out this defensive noise, the formula also incorporates a possession control ratio - a comparison of how often the player wins the ball against how often they lose it. A player who sprints aggressively while consistently retaining possession earns a boosted xT; a player who sprints just as hard but gives the ball away repeatedly has their bonus suppressed, correctly identifying their activity as defensive recovery rather than progressive threat.

Finally, the scalar applied to the whole formula varies by position. Attackers and attacking midfielders, who are primarily responsible for driving the team into dangerous areas, receive a higher base multiplier. Defensive midfielders and centre-backs - positions that sprint frequently but in predominantly reactive contexts - receive a substantially lower one. The full formula, positional scalar values, and calibration details are in the deep dive below.

Crucially, this proxy cannot scale linearly. A player increasing their passing volume from 20 to 40 accurate passes usually signifies a massive increase in active match involvement and threat. However, a player increasing from 80 to 100 passes is often just recycling safe possession in a deep defensive block. To reflect this reality, the engine applies a natural logarithm to the passing data. Doing so ensures that the xT bonus highly rewards players for establishing a baseline of dynamic progression, but applies mathematical

diminishing returns to prevent players from artificially inflating their rating through sterile, non-threatening possession.

 Deep Dive: Expected Threat (xT) Synthesis >

03a - Expected Threat Synthesis
1. The Algebraic Foundation

To synthesise a proxy for Expected Threat (xT) without spatial coordinate data, the `MatchRatingsService` uses a heuristic formula that combines a player's positional role, physical intent, possession quality, and technical execution into a single progressive threat score:

$$xT_{bonus} = S_{pos} \times C_{ctrl} \times \left(\frac{D_{sprint}}{D_{total}} \right) \times \ln(A_{pass} \times V_{pass} + 1)$$

- S_{pos} : Positional base scalar — calibrated to the role's progression responsibility.
- C_{ctrl} : Possession control ratio — a noise filter distinguishing attacking intent from defensive recovery.
- D_{sprint} : Distance sprinted per 90.
- D_{total} : Total distance covered per 90.
- A_{pass} : Pass completion percentage, expressed proportionally (0.0–1.0).
- V_{pass} : Total passes attempted per 90.

2. The Positional Scalar (S_{pos})

The base scalar is not a fixed constant. It is drawn from a position-keyed lookup at runtime, reflecting the degree to which each tactical role is expected to generate progressive attacking threat:

Tier	Positions	S_{pos}
Attacking	ST, RW, LW, CAM, CF	0.35
Mid	CM, RM, LM, RWB, LWB, RB, LB	0.25
Defensive	CDM, CB	0.10

The rationale is directly tied to positional responsibility for ball progression. Attackers and attacking midfielders are the primary drivers of progressive threat — they are expected to receive the ball in advanced areas and create danger. A winger covering a high sprint ratio with elite passing volume is doing exactly what the position demands, so the bonus is scaled generously. Conversely, a CDM or CB who sprints frequently is most likely doing so in a defensive recovery context rather than driving an attack; suppressing their scalar to 0.10 ensures the xT proxy does not misrepresent defensive activity as progressive threat. The mid-tier 0.25 was calibrated to act as a tiebreaker rather than a primary driver: if two central midfielders produce identical core statistical contributions, the engine correctly awards the marginally higher rating to the one who combined more intense physical running with higher passing volume.

For any position not found in the lookup, the engine falls back to the mid-tier default of 0.25.

3. The Physicality Multiplier and Possession Filter

The first ratio in the formula, $\frac{D_{sprint}}{D_{total}}$, establishes a player's physical intent. By expressing sprint distance as a proportion of total distance covered, the engine isolates players who spend a high

fraction of their match running at intensity — a hallmark of line-breaking runs and box-to-box dynamism — from those who cover similar total distances at a jog.

However, a high sprint ratio alone does not confirm attacking intent. A centre-back being pinned back by a sustained press may sprint aggressively and repeatedly without ever advancing the ball into a dangerous area. To filter out this defensive noise, the formula applies a possession control ratio:

$$C_{ctrl} = \min\left(\frac{P_{won} + 1}{P_{lost} + 1}, 2.5\right)$$

Where P_{won} is possession won per 90 and P_{lost} is possession lost per 90. The $+1$ Laplace smoothing in both the numerator and denominator prevents division by zero in extreme cases. The ratio is hard-capped at 2.5 to prevent an elite ball-winner from generating a disproportionate bonus purely through defensive interceptions.

A player who sprints aggressively while consistently winning the ball and rarely losing it produces a high C_{ctrl} , correctly amplifying their xT proxy. A player who sprints just as much but repeatedly turns the ball over produces a low C_{ctrl} , suppressing the bonus — their physical intensity is reflecting defensive pressure, not progressive threat.

4. The Technical Base and Logarithmic Restraint

The second half of the formula evaluates technical execution. Multiplying A_{pass} by V_{pass} calculates the exact volume of successful passes completed per 90. This product is wrapped in a natural logarithm:

$$\ln(A_{pass} \times V_{pass} + 1)$$

This applies strict mathematical diminishing returns. A player who completes 30 passes receives a significant spike, reflecting meaningful match involvement and distribution. A player who completes 90 passes receives a severely diminished marginal return, preventing sterile recycling possession around the backline from generating outsized xT. The difference between 10 and 30 successful passes is treated as far more significant than the difference between 70 and 90 — which accurately reflects the reality that the latter is more likely to represent safe, non-progressive passing.

The $+1$ constant is a structural necessity. If a player completes zero passes, $\ln(0)$ evaluates to $-\infty$, which would corrupt the rating matrix. The constant guarantees that the absolute minimum return is $\ln(1) = 0.0$, safely resolving to a zero contribution rather than an undefined value.

5. Algorithmic Implementation and Safety

```
def _calculate_xt_bonus(
    self,
    pos: str,
    pass_accuracy: float,
    passes_p90: float,
    distance_sprinted_p90: float,
    distance_covered_p90: float,
    possession_won_p90: float,
    possession_lost_p90: float,
) -> float:
    if distance_covered_p90 == 0:
        return 0.0

    base_scalar: float = self.XT_POSITION_SCALARS.get(pos, 0.25)
```

```
# Defensive noise filter (Laplace-smoothed, bounded at 2.5)
control_ratio: float = min(
    (possession_won_p90 + 1.0) / (possession_lost_p90 + 1.0), 2.5
)
return (
    base_scalar
    * control_ratio
    * (distance_sprinted_p90 / distance_covered_p90)
    * float(np.log((pass_accuracy * passes_p90) + 1.0))
)
```

The function is called only after the full Bayesian smoothing and per-90 conversion pipeline has run, ensuring that the inputs are statistically stable before the proxy is computed. Two safety guards are in place: the explicit `if distance_covered_p90 == 0` check intercepts telemetry glitches that would otherwise cause a `ZeroDivisionError` in the sprint ratio, and the `+1` inside the logarithm ensures a zero-pass performance returns 0.0 rather than crashing. Both return a clean 0.0 bonus rather than propagating an undefined value into the downstream Z-score matrix.

Live Z-Score Standardisation

Even with advanced proxies like Expected Threat, the telemetry remains dimensionally fragmented. A centre-back executing five successful tackles and a striker completing five shots on target have both registered the number 5, but the tactical weight, frequency, and difficulty of those actions will be quite different. To calculate a singular match rating that encompasses all metrics appropriately, each metric must be converted to a universally comparable unit of measurement.

Before standardisation can begin, the engine applies a logarithmic compression step to several statistics with highly skewed distributions. Metrics such as goals, shots, and several defensive counting statistics tend to cluster at zero in the vast majority of matches, with occasional extreme outlier performances. Without compression, a player who scores three goals would generate a Z-score so extreme it would overwhelm every other metric in the calculation. By applying a $\ln(x + 1)$ transform to these statistics before standardising, the engine ensures that the difference between 0 and 1 goal is treated as far more meaningful than the difference between 3 and 4. The historical baselines used for standardisation are computed on the same compressed scale, so the comparison remains mathematically consistent.

With this preprocessing applied, the Analytics Engine standardises all metrics using Z-score standardisation. Rather than evaluating the player based on raw stat volumes, the algorithm measures their performance in standard deviations from the historical positional mean ($Z = \frac{X - \mu}{\sigma}$). Conceptually, a Z-score abandons the raw data; instead of asking 'how many passes a player completed', it asks 'how mathematically exceptional was this player's passing volume compared to a typical player in their position?'. This means that players are judged relative to the positional average, giving their performances proper context.

However, standard Z-score mathematics are context-blind - they only measure distance from the mean. If a player commits five fouls in a match whilst the historical average is one, the standard formula will yield a massive positive Z-score, just as it would if the player had scored four goals. To handle this, negative telemetry stats - where the higher they are, the worse the performance (fouls committed, offsides, and possession lost) - are intercepted and handled differently. For these specific metrics, an inverted formula is used ($Z = \frac{\mu - X}{\sigma}$), ensuring that high volumes of negative actions generate heavily negative Z-scores.

Once all metrics have been run through this pipeline, instead of being expressed as raw numbers, they are decimal values grouped around 0, where the distance above or below 0 shows how much the player over- or under-performed in that metric. The majority of values will be within one standard deviation from the mean, between -1 and 1 . Stronger outlying performances may reach as far as three standard deviations. This means

all 17 metrics are now in a directly comparable form - one can look at the full array and immediately see whether a player did better or worse than expected in each statistic, with full positional context.

Volume Masking

Whilst Z-score standardisation normalises the telemetry to a comparable scale, it fails to address a fatal flaw in the efficiency metrics - pass accuracy, shot accuracy, dribble success rate, and tackle success rate. Because these metrics are percentages, they are very easily manipulated by low-volume events. A defender who completed just two passes, simple five-yard passes, would achieve a 100% pass accuracy despite their passing contribution being effectively meaningless - in no way comparable to a player with 100% pass accuracy after 40 passes. The Z-score algorithm would nonetheless treat this as a massive positive outlier, assigning an elite Z-score to a metric that is entirely uninformative.

To protect the integrity of the rating, the Analytics Engine applies a two-stage protection system to each of these four efficiency metrics. The first stage is an absolute floor: at one attempt or fewer, the efficiency Z-score is zeroed out entirely — a single pass or a single tackle carries no meaningful efficiency signal whatsoever, so the engine mutes it completely. Above this floor, the engine applies continuous logistic volume masking: rather than a further hard cut-off that simply switches trust on or off at a fixed threshold, it calculates a confidence weight between 0.0 and 1.0 based on the player's volume of attempts. Well below the threshold, the confidence weight approaches zero and the efficiency Z-score is suppressed to near nothing. Well above the threshold, the weight approaches 1.0 and the Z-score is trusted in full. In between, the weight scales smoothly — a player with borderline volume receives partial credit proportional to the statistical evidence they have accumulated.

Each efficiency metric has its own independently calibrated threshold — the point at which the confidence weight reaches exactly 50% and live data and historical expectation carry equal mathematical weight:

- **Pass accuracy:** threshold at 3 passes
- **Dribble success rate:** threshold at 2 dribbles
- **Shot accuracy:** threshold at 1.5 shots
- **Tackle success rate:** threshold at 1.5 tackles

Because the logistic transition above the absolute floor is smooth rather than binary, there is no cliff-edge where a single additional attempt suddenly unlocks a dramatically different rating. A player who attempts one more tackle than the threshold does not receive full credit at a stroke — trust accumulates gradually, and the efficiency metric only reaches its full influence once the player has demonstrated a meaningful volume of attempts.

IV. Offline Weight Generation: Dimensionality Reduction

While Z-score standardisation successfully establishes dimensional equivalence across all metrics, it introduces a secondary architectural challenge: establishing relative tactical value. A standardised passing metric and a standardised tackling metric are mathematically comparable, but these metrics do not hold equal weight in determining a player's overall match contribution. Furthermore, this valuation is inherently position-dependent; the statistical footprint of an elite centre-back is fundamentally different from that of an elite striker.

To resolve this, the Analytics Engine explicitly rejects arbitrary, hand-tuned weighting. Manually assigning tactical value introduces developer bias, inadvertently forcing the engine to judge a user's players against a rigid, subjective tactical philosophy rather than objective performance. Instead, the engine mathematically derives these weights. The algorithm executes this derivation entirely offline, leveraging a comprehensive historical dataset to calculate the weights outside of the live application environment. The resulting optimal weight matrices are then hardcoded directly into the production engine, ensuring the live software remains computationally lightweight, perfectly objective, and mathematically rigorous.

The Tangled Stats Problem

The biggest problem that comes up when attempting to derive and assign weights is the tangled stats problem: football statistics are often inherently tangled (highly correlated) with one another. For example, if a midfielder has a high number of passes, they are more likely to have more assists and a higher Expected Threat. Moreover, if a defender attempts a high number of tackles, they are also more likely to win a high number of possessions and, therefore, usually have a lower number of possessions lost. Assigning weights to these stats independently, without accounting for their relationships, creates a severe problem of multicollinearity (double- or triple-counting the exact same underlying performance trait). The engine would reward a player multiple times for doing the same thing.

The PCA Solution

To mitigate the multicollinearity hazard while maintaining tactical realism, the offline analytics pipeline utilises a hybrid mathematical approach: **Guided Principal Component Analysis (PCA)**. If standard PCA is left entirely unguided, it blindly extracts whichever metrics hold the highest historical variance. Doing so introduces a severe architectural flaw; the algorithm could theoretically assign a massive attacking weight to a Defensive Midfielder simply because shooting variance happened to be highly volatile in the dataset.

To prevent this, the engine pairs the developer's overarching tactical mandate (the Macro-Philosophy) with the algorithm's mathematical precision (the Micro-Derivation). First, the algorithm groups the 17 metrics into distinct tactical blocks: *Attacking*, *Passing*, *Dribbling*, *Defending*, *Safety*, *Work rate & Threat*, and *Tactical Discipline*. For specialist positions, the engine applies structural multipliers to these blocks *before* the PCA runs. For example, the philosophy for a Central Defensive Midfielder (CDM) artificially inflates the variance of the *Defending* and *Passing* blocks, while heavily suppressing *Work rate & Threat*. The Central Midfielder is the closest position to a neutral baseline — all football blocks carry equal philosophical weight, with only the Tactical Discipline block mildly suppressed to prevent the natural variance in fouling rates from over-influencing the output.

Each component represents a naturally occurring tactical profile defined by its own unique set of internal weights. Component 1 always captures the largest chunk of variance (e.g., 45%) and maps to the dominant trait of that position, while Component 2 captures the next largest chunk (e.g., 20%) for secondary traits. Because these components are mathematically orthogonal — meaning they have zero statistical correlation with one another — the algorithm successfully isolates a player's offensive output from their defensive output.

Finally, the engine must collapse these independent dimensions back into a single, usable set of multipliers. To generate the final positional weight for a specific stat, the algorithm looks at that stat's loading across all retained components, scales each loading by the square root of the variance that component captures, and sums them. This ensures the stats that define the dominant tactical patterns of a position — and that appear most prominently across the most influential components — receive the highest weights.

Derived Position Weights

Three positions — Wingbacks (LWB/RWB), Wide Midfielders (RM/LM), and Central Attacking Midfielders (CAM) — had insufficient match data in the historical dataset to support a reliable standalone PCA run. Rather than introducing synthetic data or forcing a PCA on a sample too small to produce stable components, the engine derives their weights through a principled inheritance process. Each borrows the weight vector of its closest tactical relative — Wingbacks from Fullbacks, Wide Midfielders from Wingers, CAMs from Central Midfielders — and applies a set of role-specific multipliers that shift the emphasis toward the unique demands of the position, before renormalising to sum to 1.0. Full methodology is in the deep dive below.

 [Deep Dive: Principal Component Analysis and Weight Generation >](#)

04b - PCA Algorithm and Generating Weights

1. Dataset Preprocessing

Before any PCA mechanics run, the historical dataset undergoes three preprocessing steps specific to the offline calibration pipeline. These steps do not occur in the live scoring engine — they are applied once during development to ensure the weight derivation reflects clean, context-adjusted football performance rather than raw counting artefacts.

Possession Adjustment

A consistent bias exists in any dataset drawn from a single team: a possession-dominant side will see their players accumulate higher passing and dribbling counts simply because the team holds the ball more, not because any individual is performing better. Conversely, defensive stats like tackles and possession won are naturally suppressed for high-possession teams because the opposition rarely has the ball long enough to lose it. Left unadjusted, PCA would derive weights that systematically over-reward the ball-dominant actions of possession-heavy teams and under-reward the defensive actions that define lower-block football.

To neutralise this bias, every volume stat is adjusted to a 50% possession baseline before Bayesian smoothing is applied:

$$X_{padj} = X_{raw} \times \sqrt{\frac{50}{P_{team}}} \quad (\text{attacking stats})$$
$$X_{padj} = X_{raw} \times \sqrt{\frac{50}{100 - P_{team}}} \quad (\text{defensive stats})$$

Where P_{team} is the team's possession percentage in that match. Attacking stats adjusted include passes, dribbles, shots, and possession lost; defensive stats adjusted include tackles, possession won, and fouls committed. The square root softens the correction so that genuinely exceptional performances on possession-extreme teams are not over-penalised — only the systemic possession bias is dampened, not the individual quality signal.

Non-Goal Shots Derivation

Before the per-90 metrics enter the Z-score and PCA pipeline, a 17th column is derived from the shot data. Rather than using total shots per 90, the engine isolates non-goal shots:

$$non_goal_shots_p90 = \max(shots_p90 - goals_p90, 0)$$

This separates pure chance creation from goal-scoring output. Without this split, a striker who scored twice from two shots would receive a high Z-score for both goals and shots — double-counting the same two events. The separation ensures goals are evaluated as goals and the shot metric captures only the chance-generation component.

Log-Normal Transformation

Several statistics have heavily right-skewed distributions: goals, assists, shots, and certain defensive counting stats are zero in the majority of match observations, with rare extreme values. Applying Z-scores directly to these raw values would allow outlier performances to exert disproportionate influence over the PCA covariance structure — a player scoring four goals in a single match could warp the entire weight derivation for a position.

To compress these distributions before standardisation, a $\ln(x + 1)$ transform is applied to the following columns in the offline notebook:

```
goals_p90, assists_p90, non_goal_shots_p90, offsides_p90, fouls_committed_p90,
possession_won_p90, possession_lost_p90
```

The means and standard deviations stored for these columns — used both to Z-score the PCA inputs and as reference baselines for live scoring — are computed from the log-transformed values. This is why the live engine applies a matching $\ln(x + 1)$ compression before Z-scoring these same stats at inference time, and why their stored means must be back-converted via `expm1` before use as Bayesian priors (as described in Section II).

2. Tactical Block Scaling

With the preprocessed dataset ready, the engine must structure it to account for two distinct architectural hazards before the covariance matrix is computed: structural size bias and blind variance optimisation.

The 17 metrics are grouped into tactical blocks. Without scaling, blocks with more features (e.g., *Attacking* has 4 stats) artificially inject more variance into the dataset than smaller blocks (e.g., *Safety* has 1 stat), biasing the PCA to favour large blocks regardless of tactical importance. Furthermore, if standard PCA is left entirely unguided, it will blindly extract whichever metrics hold the highest variance. A CDM dataset may show high variance in shooting simply because in some matches they are pinned back while in others they roam forward — standard PCA would assign shooting a high weight not because it matters for a CDM, but because it happens to fluctuate.

To resolve both hazards, the engine applies a **Guided Variance Transformation**. The developer defines a macro-philosophy dictionary assigning a structural multiplier to each block based on the positional mandate. The standard block schema shared by most positions is:

```
blocks = {
  "Attacking": [
    "goals_p90_z", "assists_p90_z", "non_goal_shots_p90_z", "shot_accuracy_z"
  ],
  "Passing": [
    "passes_p90_z", "pass_accuracy_z"
  ],
  "Dribbling": [
    "dribbles_p90_z", "dribble_success_rate_z"
  ],
  "Safety": [
    "possession_lost_p90_z"
  ],
  "Defending": [
    "tackles_p90_z", "tackle_success_rate_z", "possession_won_p90_z"
  ],
  "Workrate_and_Threat": [
    "distance_covered_p90_z", "distance_sprinted_p90_z", "xt_bonus_p90_z"
  ],
  "Tactical_Discipline": [
    "offsides_p90_z", "fouls_committed_p90_z"
  ],
}
```

Each position then defines its own philosophy dictionary. Two examples — the CDM as a specialist position and the CM as the near-neutral baseline:

```
# CDM: heavily emphasises defending and passing, suppresses dribbling, mildly
reduces workrate
cdm_philosophy = {
    "Attacking": 1.0,
    "Passing": 2.7,
    "Dribbling": 0.6,
    "Safety": 1.4,
    "Defending": 2.5,
    "Workrate_and_Threat": 0.8,
    "Tactical_Discipline": 1.0,
}

# CM: all football blocks equal; Tactical Discipline mildly suppressed
cm_philosophy = {
    "Attacking": 1.0,
    "Passing": 1.0,
    "Dribbling": 1.0,
    "Safety": 1.0,
    "Defending": 1.0,
    "Workrate_and_Threat": 1.0,
    "Tactical_Discipline": 0.5,
}
```

The Tactical Discipline suppression for CM reflects a calibration finding: fouling rate has high natural variance in the training data but is a weak differentiator between CM performances in practice. Leaving it at 1.0 allows noise to over-influence the PCA output.

When iterating through the Z-scores, the engine identifies the number of stats within each block (k) and queries the philosophy dictionary for the assigned multiplier W_{block} . Every stat in the block is scaled by:

$$X_{scaled} = X_z \times \frac{W_{block}}{\sqrt{k}}$$

Dividing by $\sqrt{k}$ establishes a neutral size baseline — the 4-stat *Attacking* block does not mathematically overpower the 1-stat *Safety* block simply by having more columns. Multiplying by W_{block} then artificially dilates or compresses the variance of specific tactical zones before the PCA analyses the matrix, steering the eigen-decomposition toward the intended positional priorities.

3. Robust Covariance Estimation and the MCD Algorithm

Standard PCA relies on a traditional sample covariance matrix. However, the standard formula is highly sensitive to extreme outliers because it calculates variance using the arithmetic mean of the entire dataset — a single anomalous performance (a goalkeeper playing outfield, a red card in the third minute) can warp the covariance structure and produce weight vectors that do not generalise to typical matches.

To establish algorithmic stability, the engine explicitly rejects the standard sample covariance in favour of the **Minimum Covariance Determinant (MCD)** estimator.

Visualise the dataset as a matrix where every row is a single match performance. Standard covariance uses every row. The MCD algorithm, configured with `support_fraction=0.98`, is instructed to deliberately discard 2% of rows to find the cleanest possible core dataset. Mathematically, H denotes this clean subset. Because the algorithm does not know in advance which rows are anomalies, it searches by evaluating thousands of different combinations of 98% subsets. For every candidate H , it computes the determinant of the associated covariance matrix C_H . Geometrically, a lower determinant means the data points are packed more tightly together:

$$\min_H \det(C_H)$$

The combination that yields the smallest determinant is the tightest, most consistent cluster of match performances — the 2% of rows causing the data ellipsoid to stretch are the anomalies and are discarded. The robust covariance matrix is then computed exclusively from the surviving rows:

$$C_{MCD} = \frac{1}{h-1} \sum_{i \in H} (x_i - \mu_{MCD})(x_i - \mu_{MCD})^T$$

```
mcd = MinCovDet(random_state=42, support_fraction=0.98).fit(scaled_pca_df)
cov = mcd.covariance_
```

Substituting C_{MCD} in place of the standard covariance ensures the downstream eigen-decomposition is built strictly on sustainable, repeatable football logic rather than being warped by statistical anomalies.

4. Eigen-Decomposition and Dimensional Truncation

With the robust covariance matrix established, the algorithm executes eigen-decomposition — a linear algebra operation that rotates the coordinate system to align exactly with the axes of maximum variance in the dataset:

$$C_{MCD} \mathbf{v}_i = \lambda_i \mathbf{v}_i$$

For a 17×17 covariance matrix, this extracts 17 pairs:

- **Eigenvectors ($\mathbf{v}_i$):** Directional vectors. In football terms, each represents a distinct tactical profile — e.g., "Defensive Dominance" or "Passing Range." The values inside dictate the recipe of stats that define that profile.
- **Eigenvalues (λ_i):** Magnitudes. Each eigenvalue defines exactly how much variance its corresponding tactical profile explains in the dataset.

The engine sorts components in descending order by eigenvalue and applies dimensional truncation, retaining only the top $k = 9$ components. These account for approximately 85% of the dataset's total tactical variance. The remaining components are discarded because they represent statistical noise — highly specific, mathematically unstable correlations that do not consistently define player performance across different matches and conditions.

5. Vector Reassembly, Scale Correction, and Normalisation

At this stage the engine holds a 17×9 matrix of eigenvectors across 9 retained components. It must collapse this back into a single 1D positional weight vector.

For every stat j , the engine aggregates its loading across all retained components, scaling each loading by the square root of the corresponding eigenvalue:

$$w_{raw,j} = \sum_{i=1}^k |v_{j,i}| \sqrt{\lambda_i}$$

Two mathematical decisions here are paramount:

1. **Absolute Values ($|v|$):** The sign of an eigenvector is mathematically arbitrary — it only indicates direction along the axis, not importance. Taking the absolute value ensures the algorithm captures the true magnitude of each stat's influence on the tactical profile regardless of sign.
2. **Square Root Scaling ($\sqrt{\lambda}$):** Eigenvalues represent variance. Taking the square root converts them to standard deviation, which accurately reflects the component's physical spread in the data space and gives proportionally more weight to the components that explain the most variance.

```
weighted_loadings = np.abs(top_eigenvectors) * np.sqrt(top_variances)
raw_weights = np.sum(weighted_loadings, axis=1)
```

Post-Hoc Scale Correction

The block scaling in Section 2 transformed the input space: each stat was divided by `scale_factor = sqrt(k) / W_block` before the PCA ran. This means the eigenvectors live in the scaled space, not the raw Z-score space — they were calibrated against amplified or suppressed stats, not the original standardised values. At inference time the live engine applies weights to raw Z-scores, so a correction is required.

For each stat, the raw weight is divided by the same scale factor that was applied during preprocessing, then the full vector is renormalised:

$$w_{corrected,j} = \frac{w_{raw,j}}{s_j} \quad \text{where } s_j = \frac{\sqrt{k_{block(j)}}}{W_{block(j)}}$$

$$w_{final,j} = \frac{w_{corrected,j}}{\sum_j w_{corrected,j}}$$

```
# Build scale factor array matching column order
col_order = scaled_pca_df.columns.tolist()
scale_factors_arr = np.ones(len(col_order))
for block_name, stats in blocks.items():
    k = len(stats)
    sf = np.sqrt(k) / philosophy[block_name]
    for stat in stats:
        scale_factors_arr[col_order.index(stat)] = sf

corrected_weights = raw_weights / scale_factors_arr
final_weights = corrected_weights / corrected_weights.sum()
```

The intuition: a stat in a heavily inflated block (e.g., Defending for a CB at $W_{block} = 3.0$) has a small `scale_factor` ≈ 0.577 , meaning it was amplified roughly 1.73 $\times$ before PCA. The raw weights from PCA reflect this amplification. Dividing by the small scale factor increases the corrected weight proportionally, correctly restoring the intended tactical emphasis in the raw Z-score space. Without this step, the final weights would be miscalibrated — they would appear to reflect the tactical philosophy but would under-deliver it when applied to unscaled live data.

The final normalised weight vector sums to 1.0 and accurately reflects the multidimensional tactical philosophy for the position without double-counting the underlying telemetry.

6. Derived Position Weights

Three positions — Wingbacks (LWB/RWB), Wide Midfielders (RM/LM), and Central Attacking Midfielders (CAM) — had insufficient match data in the historical dataset to support a reliable standalone PCA run. Running PCA on a small or unrepresentative sample risks producing unstable components whose loadings shift dramatically with minor dataset changes, making the derived weights unreliable. Introducing synthetic data carries its own calibration risks. Instead, these positions use a principled inheritance approach.

Each borrows the final weight vector of its closest tactical relative as a starting point:

Derived Position	Proxy Base
Wingback (LWB/RWB)	Fullback (RB)
Wide Midfielder (RM/LM)	Winger (RW)
Central Attacking Midfielder (CAM)	Central Midfielder (CM)

A set of per-stat multipliers is then applied to shift the emphasis toward the unique demands of the derived role. For example, Wingbacks inherit from Fullbacks but are expected to contribute far more in the final third — so their attacking and dribbling weights are scaled up, their defensive weights scaled down, and their sprint and xT weights scaled up to reflect higher physical and progressive involvement. Wide Midfielders inherit from Wingers but with heavier defensive duties, so tackling and possession won are upweighted. CAMs inherit from CMs with elevated attacking contribution and suppressed defensive involvement.

The full set of multipliers for each derived position is defined in the offline calibration notebook. After multipliers are applied, the weight vector is renormalised to sum to 1.0:

```
derived_weights = {}
for stat, base_weight in proxy_weights.items():
    multiplier = position_multipliers.get(stat, 1.0)
    derived_weights[stat] = base_weight * multiplier

total = sum(derived_weights.values())
derived_weights = {k: v / total for k, v in derived_weights.items()}
```

The means and standard deviations used for Z-scoring these positions at inference time are derived from the proxy position's calibration data, adjusted using the same scaling assumptions. This is a deliberate approximation — the true positional distribution for these roles would ideally come from a dedicated dataset, but the tactical proximity of the base positions makes the inherited baselines a reasonable and stable substitute.

V. The Scoring Algorithm

With the live match telemetry mathematically standardised into contextual Z-scores (Section III) and the ideal tactical philosophies distilled into independent Positional Weight Vectors (Section IV), the Analytics Engine reaches its core synthesis phase.

At this stage, the live application dynamically queries the hardcoded PCA weight matrices and aligns them with the player's live performance data. By calculating the mathematical dot product of these two arrays — multiplying every individual standardised metric by its mathematically derived tactical weight and summing the result — the engine compresses 17 disparate dimensions of telemetry into a singular, unbounded raw performance score.

However, an unbounded raw score (e.g., +4.82 or -1.75) is unreadable to an end user who requires the traditional 1.0 to 10.0 football rating scale. Furthermore, a purely statistical dot product lacks the macro-context of the match result. The engine cannot award a defender who executes flawless tackles a perfect rating if the team simultaneously concedes four goals.

To convert this raw synthesis into a mathematically bounded, contextually aware final rating, the engine passes the score through a four-step pipeline: positional context modification, impact scaling, logistic (sigmoid) transformation, and macro-supremacy scaling. The first three are covered in this section; macro-supremacy scaling is addressed in Section VI.

Positional Modifiers & Contextual Heuristics

The foundational step of the scoring algorithm is the dot product: multiplying the 17 standardised Z-scores by their respective positional weights to generate a base score. However, while a dot product is perfectly linear, football is inherently non-linear and highly contextual. A purely statistical synthesis cannot account for the asymmetrical responsibilities of different tactical roles, nor can it comprehend the macro-state of the match. To bridge the gap between pure mathematics and football reality, the raw score is passed through a pipeline of positional modifiers.

This modification pipeline executes four distinct architectural layers:

1. **Asymmetric Flooring:** Standardised Z-scores mathematically punish players for inaction. The engine intercepts this by applying positional floors. For example, a Centre Back is granted an "Attacking Floor" that neutralises negative Z-scores for a lack of shots or assists, treating their offensive absence as a neutral baseline rather than a mathematical detriment. Conversely, a Winger receives "Defensive Absolution" — a floor that prevents a lack of tackling from generating a heavy penalty.
2. **Efficiency Safeguards:** Percentage-based metrics (like pass accuracy) can generate massive negative Z-scores from tiny sample sizes — a player who completes 1 of 3 passes would otherwise receive an extreme penalty. The engine floors these efficiency Z-scores at a standard penalty value, ensuring a poor success rate in isolated actions registers as a "bad day" rather than an unrecoverable mathematical black hole.
3. **Absolute Output Multipliers & Build-Up Isolation:** Z-score standardisation smooths variance, but goals and assists are absolute, match-defining events. These are applied as efficiency-adjusted bonuses after the dot product — a player who scores from fewer shots earns a larger bonus per goal, since they converted chances that statistically should not have gone in, while a per-goal floor ensures scoring always contributes positively regardless of shot volume. Before these bonuses are awarded, the engine evaluates whether the player was genuinely involved in the match's build-up. A player whose passing, dribbling, and work-rate Z-scores collectively indicate they were largely absent from the flow of the game has their goal and assist bonuses exponentially decayed. A player who ghosted through the game and then scored receives a meaningfully reduced bonus compared to one who was consistently involved before earning their reward.
4. **Contextual Heuristics:** The engine deploys position-specific logic to evaluate how a player accumulated their stats. It calculates whether a striker acted as an offensive "black hole" by comparing turnover rate to positive involvements, rewards a midfielder who contributed to keeping a clean sheet with a flat bonus, and dynamically scales a defender's clean sheet reward based on the quality of chances the opponent actually created.

Following the positional modifier pipeline, an **impact scalar** is applied to the accumulated base score before the sigmoid runs. Goal and assist bonuses are then added as a separate, unscaled term — so a substitute who scores still receives the full credit for that goal, never diminished by their brief appearance. Equal to the square root of the player's effective minutes as a fraction of 90 — for example, $\sqrt{45/90} \approx 0.71$ for a half-match substitute — this compresses abbreviated base performances toward the 6.0 baseline at the output stage. It acts as a second insurance layer alongside the Bayesian smoothing applied at the input stage in Section II:

Bayesian smoothing constrains individual per-90 rates; the impact scalar compresses the entire synthesised base score.

Deep Dive: The Synthesis Pipeline & Contextual Heuristics >

05a - Synthesis Pipeline & Contextual Logic

Phase I: Pre-Synthesis Matrix Bounding

Before the engine calculates the baseline match rating, it must conditionally bound the raw Z-score input vector to prevent isolated data points from collapsing the linear algebra.

The engine iterates through the 17-element telemetry array and applies minimum thresholds depending on the player's position:

- **Efficiency Safeguards:** Percentage-based metrics — pass accuracy, shot accuracy, dribble success rate, and tackle success rate — are regulated by the continuous logistic volume masking system applied during Z-score standardisation (Section III). By the time the dot product runs, these Z-scores already carry a confidence weight proportional to the player's attempt volume; a player with near-zero attempts contributes almost nothing to the dot product through their efficiency score regardless of how extreme that score might be on a raw basis. Any position-specific hard floors on individual efficiency metrics are documented in Appendix A.
- **Tactical Instruction Floors:** To protect players from being penalised for following managerial orders, the engine applies contextual forgiveness by role. A Fullback instructed to stay back will naturally log low dribbling volume and low xT — the engine intercepts these specific metrics and floors them at a modest penalty (-0.50), ensuring a defensive fullback is not mathematically destroyed for ignoring the final third. Similar absolution is applied to attacking positions for below-average defensive metrics, and to defensive positions for below-average attacking metrics.

Phase II: Base Synthesis (The Dot Product)

With the input vector safely bounded, the raw foundation of the match rating is calculated using the dot product of the modified Z-scores and the Positional Weight Vector:

$$S_{raw} = \sum_{i=1}^{17} (Z_i \times W_i)$$

```
# Fast, vectorized linear combination of the 17 core metrics
return np.dot(
    np.array([z_scores.get(f"{col}_z", 0) for col in col_names]),
    weights,
)
```

Phase III: Post-Synthesis Contextual Heuristics

Following the dot product, the engine applies non-linear modifiers to S_{raw} to account for complex match context, output rewards, and dynamic player archetypes.

Tactical Build-Up Isolation

Before goal and assist bonuses are applied, the engine evaluates whether the player was genuinely involved in the match's build-up phase. It averages the Z-scores across six build-up metrics —

passes, pass accuracy, dribbles, dribble success rate, distance covered, and distance sprinted — to form a composite build-up score z_{build} .

If $z_{build} < -1.0$, the player's build-up contribution is classified as statistically absent. Rather than a binary cut-off, the engine calculates an exponential decay multiplier:

$$\text{isolation_multiplier} = \begin{cases} e^{z_{build}+1.0} & \text{if } z_{build} < -1.0 \\ 1.0 & \text{otherwise} \end{cases}$$

This multiplier is applied to all goal and assist bonuses in the subsequent step. A player at $z_{build} = -1.5$ receives a multiplier of $e^{-0.5} \approx 0.61$, reducing their bonuses to 61% of face value. A player at $z_{build} = -2.5$ receives $e^{-1.5} \approx 0.22$, effectively stripping most of the bonus for a player who was almost entirely absent from the game. The multiplier only decays when build-up involvement is meaningfully negative — a mildly quiet game at $z_{build} = -0.9$ leaves the full bonus intact.

Bottleneck Synergy Bonuses

Rather than forcing players into a single rigid positional mould, the engine dynamically rewards distinct archetypes within the same position based on statistical output. These bonuses activate and scale continuously rather than applying a flat reward at a binary threshold.

The core mechanic is **bottleneck synergy**: a bonus only activates when a player simultaneously exceeds a threshold across two related metrics, and the bonus rate is driven by the weaker of the two. This prevents one extraordinary stat from generating a bonus while masking a deficiency in the paired metric. For example:

- **Fullback "Third CB" Archetype**: Activates when both `tackles_z` and `possession_won_z` exceed 1.0. The bonus scales with $\min(\text{tackles_z}, \text{poss_won_z}) - 1.0$.
- **Fullback "Express Train" Archetype**: Activates when both `distance_sprinted_z` and `xt_bonus_z` exceed 1.0. The bonus scales with $\min(\text{sprint_z}, \text{xt_z}) - 1.0$.

Analogous bottleneck bonuses exist for every outfield position — a CDM "Destroyer" archetype gated on tackles and possession won, a CM "Progression Engine" gated on passes and dribbles, a CAM "Maestro" gated on passes and xT, and so on — each reflecting the paired tactical requirements of the role.

Absolute Output Multipliers

Goal contributions are applied as efficiency-adjusted bonuses, modulated by the isolation multiplier calculated above. Rather than a flat reward per goal, the engine calculates an *above-expected* component: the number of goals minus a crude estimated xG based on shot volume (0.20 xG per shot). A floor of 40% of the position-specific coefficient per goal ensures scoring always contributes positively, even for a high-volume shooter:

$$\begin{aligned} \text{above_expected} &= \max(\text{goals} - \text{shots} \times 0.20, \text{goals} \times 0.40) \\ \text{event_bonus} &= (\text{above_expected} \times G_{pos} + \text{assists} \times A_{pos}) \times \text{isolation_multiplier} \end{aligned}$$

Where G_{pos} and A_{pos} are the position-specific goal and assist coefficients. These vary by role — a Striker's goal carries a coefficient of 1.5, a Centre Back's 0.75, and a CDM's 0.6 — reflecting how central scoring is to each position's mandate. The complete per-position values are in Appendix A.

This `event_bonus` is computed here but held separately from S_{raw} ; it is not folded into the base score until after the impact scalar in Phase IV.

Contextual Performance Gates

The engine applies additional macro-contextual rewards and penalties specific to each position. Examples across the pipeline:

- **Dynamic Clean Sheet Bonus (Defenders):** Scaled by opponent xG. A clean sheet against ≤ 1.0 xG yields a maximum $+0.50$ reward, reflecting genuine defensive dominance. Against ≥ 2.0 xG it yields a minimal $+0.15$, acknowledging the result may owe as much to poor opponent finishing or exceptional goalkeeping as to outfield defending.
- **Collapse Penalty (Defenders):** A flat -0.30 is applied if the team concedes three or more goals and the player was on the pitch for at least 60 minutes.
- **Wasteful Finisher Penalty (Striker):** Assigns 0.20 xG per shot. If a striker takes more than 3 shots and generates a finishing deficit above 0.75 , a scaling penalty capped at -0.80 is applied.
- **Reliable Pivot Bonus (CDM):** If 60+ minutes are played with pass accuracy $\geq 92\%$ and elite passing volume, a tiered bonus rewards near-perfect ball retention: $+0.35$ for zero possession lost ("Perfect Metronome"), $+0.20$ for at most one loss ("Reliable Shift").
- **Clean Sheet Bonus (Midfielders):** Central and defensive midfielders who complete 60+ minutes with zero goals conceded receive a flat $+0.15$, reflecting their role in screening the defensive line.

Phase IV: Impact Scaling

After all positional modifiers and heuristics have been applied to the base score, the engine scales it and combines it with the goal contribution to form the final pre-sigmoid input:

$$S_{final} = S_{raw} \times \sqrt{\frac{\min(M, 90)}{90}} + \text{event_bonus}$$

Where S_{raw} is the accumulated dot-product score including all archetype bonuses, contextual heuristics, and positional penalties from Phase III, and event_bonus is the goal and assist contribution computed in Phase III. Only S_{raw} passes through the scaling step; the event bonus is added unscaled afterwards.

This separation is a deliberate design decision. Compressing a goal bonus by the fraction of the match a substitute played would undervalue an absolute, match-defining contribution — a player who comes on and scores should receive the full credit for that goal. The impact scalar instead compresses the base statistical performance toward zero, correctly reflecting that a brief appearance does not provide sufficient evidence to award the same base rating as a full-match performance, while leaving the factual output intact.

Where M is the player's actual minutes played. A full 90-minute match yields a multiplier of exactly 1.0 and S_{raw} passes through unchanged. A player with 45 minutes receives ≈ 0.71 ; a player with 22 minutes receives ≈ 0.49 .

The rationale is distinct from Bayesian smoothing. Smoothing (Section II) constrains the per-90 rates fed into the Z-score calculation — it operates at the input stage and affects how each individual metric is evaluated. The impact scalar operates at the output stage, compressing the entire synthesised base score toward zero regardless of which individual metrics drove it. A 22-minute substitute who generated a strong $+2.0$ base score has it compressed to roughly $+0.99$ before the event bonus is added and the sigmoid runs, producing a final rating comfortably above average but well short of what a full-match performer with the same statistical intensity would earn.

Logistic Mapping

After the positional modifiers and impact scaling are applied, the engine has a single raw score for the player's performance. Because this score is built from standard deviations, it looks like a bizarre decimal — a terrible game might be -2.5 , an average game $0.00.0\ 0.0$, and a world-class game $+4.8$. This format is meaningless to an end-user. The application must convert it into the classic 1.0 to 10.0 match rating scale used by *FIFA*, *WhoScored*, and football fans worldwide.

A naïve approach would be to scale the numbers evenly — mapping the highest historical score to a 10 and the lowest to a 1. However, football throws up crazy outliers. If a player scores six goals in a single game, an even scale would permanently warp the system, compressing every other performance into an unusable range.

To safely lock the ratings between 1 and 10 without breaking the scale, the Analytics Engine uses an S-Curve (mathematically known as a Sigmoid transformation).

Instead of stretching numbers evenly, the curve is anchored to a psychological baseline: an exactly average statistical performance is locked to a **6.0 rating**. In the middle of the curve — where the vast majority of football matches sit — the scoring is highly sensitive, easily separating a decent 6.5 performance from a great 7.5. At the extreme top end the curve applies diminishing returns: a world-class hat-trick might earn a 9.5, and adding a fourth goal might only push it to a 9.8.

Crucially, the S-curve is deliberately asymmetric. Above the 6.0 baseline it climbs steeply, providing clear separation between good, great, and elite. Below it the curve falls more gradually — a poor performance settles naturally in the 4.0 to 5.5 range rather than plunging toward 1.0, reflecting the reality that ratings below 4.0 are reserved for genuinely catastrophic matches rather than merely average ones. The full mathematical derivation of both curves is in the deep dive below.

Deep Dive: Logistic Mapping Mathematics >

05b - Logistic Mapping Mathematics

1. The Asymmetric Logistic Function

To map the unbounded continuous variable $S_{final} \in (-\infty, \infty)$ to the recognised 0.0–10.0 continuum, the engine uses a scaled logistic function with a steepness parameter that branches on the sign of the input:

$$R_{raw} = 10 \times \frac{1}{1 + e^{-k(S_{final} - S_0)}}$$
$$k = \begin{cases} 0.85 & \text{if } S_{final} \geq 0 \\ 0.45 & \text{if } S_{final} < 0 \end{cases}$$

- R_{raw} : The resulting 0–10 match rating, before supremacy scaling.
- S_{final} : The impact-scaled raw score from the positional modifier pipeline.
- k : The steepness parameter, controlling the slope of the S-curve on each side of the baseline.
- S_0 : The translation parameter, derived from k to lock the median at exactly 6.0.

The asymmetry is deliberate: the positive side climbs steeply to differentiate between good, great, and elite performances, while the negative side descends more gently to reflect the psychological reality that football ratings below 4.0 are reserved for genuinely catastrophic matches rather than merely below-average ones.

2. Anchoring the Baseline (S_0 and the 6.0 Median)

In a pure mathematical model mapped from 0 to 10, the median defaults to 5.0. However, the Analytics Engine explicitly rejects a 5.0 median because it violates the psychological reality of football

ratings.

In established industry frameworks (e.g., *L'Equipe*, WhoScored), ratings possess a natural positive skew. A player who performs terribly rarely receives below a 4.0, whereas excellent players frequently receive 8.0+. To account for this asymmetric headroom, the engine locks an entirely average, 0.0 standard deviation performance to exactly **6.0**.

The derivation of S_0 is identical regardless of which value of k is active. Setting $S_{final} = 0$ and solving for $R_{raw} = 6.0$:

$$\begin{aligned} 6.0 &= 10 \times \frac{1}{1 + e^{k \cdot S_0}} \\ 1 + e^{k \cdot S_0} &= \frac{10}{6} \implies e^{k \cdot S_0} = \frac{2}{3} \\ k \cdot S_0 &= \ln\left(\frac{2}{3}\right) \approx -0.405 \\ S_0 &= \frac{\ln(2/3)}{k} \end{aligned}$$

Because S_0 is a function of k , both steepness values produce curves that pass through exactly 6.0 at $S_{final} = 0$. The 6.0 baseline is guaranteed regardless of which side of zero the input falls on.

3. Calibrating the Steepness Parameters

The Positive Side ($k = 0.85$)

Above the 6.0 baseline, k governs how quickly an excellent performance climbs toward a perfect 10.0.

If k is set too high (e.g., $k = 1.5$), the curve is too steep: a standard great game at $+2.5\sigma$ would immediately hit 10.0, making the perfect rating far too common and eliminating the distinction between very good and genuinely exceptional. If k is too low (e.g., $k = 0.4$), the curve is too flat: a historic, multi-goal performance at $+4.0\sigma$ would struggle to surpass 8.5, unfairly punishing genuine brilliance.

$k = 0.85$ is the calibrated equilibrium. Under this slope, a strongly anomalous $+3\sigma$ performance organically approaches 9.5. Reaching a true 10.0 requires pushing beyond the normal limits of the statistical distribution — possible, but demanding. The corresponding translation parameter is:

$$S_0 = \frac{\ln(2/3)}{0.85} \approx -0.477$$

The Negative Side ($k = 0.45$)

Below the 6.0 baseline, a symmetric $k = 0.85$ would map a -2.5σ performance to approximately 3.3 — a rating psychologically reserved for catastrophic collapses: goalkeepers conceding five, defenders scoring own goals, strikers invisible for 90 minutes. A player who simply had a poor but unremarkable statistical match should settle in the 4.5 to 5.5 range, not plunge toward the floor.

$k = 0.45$ produces this gentler descent. Under this slope, a -2.5σ performance maps to approximately 4.3 and a -3σ performance to approximately 3.9. The floor remains reachable for genuinely extreme negative output, but ordinary bad games are treated proportionately. The corresponding translation parameter is:

$$S_0 = \frac{\ln(2/3)}{0.45} \approx -0.900$$

4. Algorithmic Implementation

The Python implementation explicitly calculates S_0 at runtime from k using `np.log`, preventing floating-point rounding errors that could shift the 6.0 baseline over thousands of iterations. The asymmetric branch is resolved first, so every evaluation uses a self-consistent (k, S_0) pair:

```
def _apply_sigmoid_transformation(self, raw_score: float) -> float:
    k = 0.85 if raw_score >= 0 else 0.45
    s_0: float = np.log(2 / 3) / k
    return 10 * (1 / (1 + np.exp(-k * (raw_score - s_0))))
```

The result is a 6.0-anchored curve that climbs steeply through the positive range while descending gently through the negative — matching the psychological distribution of football ratings as understood by fans, managers, and analysts alike.

Logistic Mapping

After the positional modifiers and impact scaling are applied, the engine has a single raw score for the player's performance. Because this score is built from standard deviations, it looks like a bizarre decimal — a terrible game might be -2.5 , an average game 0.0 , and a world-class game $+4.8$. This format is meaningless to an end-user. The application must convert it into the classic 1.0 to 10.0 match rating scale used by *FIFA*, *WhoScored*, and football fans worldwide.

A naïve approach would be to scale the numbers evenly — mapping the highest historical score to a 10 and the lowest to a 1. However, football throws up crazy outliers. If a player scores six goals in a single game, an even scale would permanently warp the system, compressing every other performance into an unusable range.

To safely lock the ratings between 1 and 10 without breaking the scale, the Analytics Engine uses an S-Curve (mathematically known as a Sigmoid transformation).

Instead of stretching numbers evenly, the curve is anchored to a psychological baseline: an exactly average statistical performance is locked to a **6.0 rating**. In the middle of the curve — where the vast majority of football matches sit — the scoring is highly sensitive, easily separating a decent 6.5 performance from a great 7.5. At the extreme top end the curve applies diminishing returns: a world-class hat-trick might earn a 9.5, and adding a fourth goal might only push it to a 9.8.

Crucially, the S-curve is deliberately asymmetric. Above the 6.0 baseline it climbs steeply, providing clear separation between good, great, and elite. Below it the curve falls more gradually — a poor performance settles naturally in the 4.0 to 5.5 range rather than plunging toward 1.0, reflecting the reality that ratings below 4.0 are reserved for genuinely catastrophic matches rather than merely average ones. The full mathematical derivation of both curves is in the deep dive below.

Deep Dive: Logistic Mapping Mathematics >

05b - Logistic Mapping Mathematics

1. The Asymmetric Logistic Function

To map the unbounded continuous variable $S_{final} \in (-\infty, \infty)$ to the recognised 0.0–10.0 continuum, the engine uses a scaled logistic function with a steepness parameter that branches on the sign of the input:

$$R_{raw} = 10 \times \frac{1}{1 + e^{-k(S_{final} - S_0)}}$$

$$k = \begin{cases} 0.85 & \text{if } S_{final} \geq 0 \\ 0.45 & \text{if } S_{final} < 0 \end{cases}$$

- R_{raw} : The resulting 0–10 match rating, before supremacy scaling.
- S_{final} : The impact-scaled raw score from the positional modifier pipeline.
- k : The steepness parameter, controlling the slope of the S-curve on each side of the baseline.
- S_0 : The translation parameter, derived from k to lock the median at exactly 6.0.

The asymmetry is deliberate: the positive side climbs steeply to differentiate between good, great, and elite performances, while the negative side descends more gently to reflect the psychological reality that football ratings below 4.0 are reserved for genuinely catastrophic matches rather than merely below-average ones.

2. Anchoring the Baseline (S_0 and the 6.0 Median)

In a pure mathematical model mapped from 0 to 10, the median defaults to 5.0. However, the Analytics Engine explicitly rejects a 5.0 median because it violates the psychological reality of football ratings.

In established industry frameworks (e.g., *L'Equipe*, WhoScored), ratings possess a natural positive skew. A player who performs terribly rarely receives below a 4.0, whereas excellent players frequently receive 8.0+. To account for this asymmetric headroom, the engine locks an entirely average, 0.0 standard deviation performance to exactly **6.0**.

The derivation of S_0 is identical regardless of which value of k is active. Setting $S_{final} = 0$ and solving for $R_{raw} = 6.0$:

$$6.0 = 10 \times \frac{1}{1 + e^{k \cdot S_0}}$$

$$1 + e^{k \cdot S_0} = \frac{10}{6} \implies e^{k \cdot S_0} = \frac{2}{3}$$

$$k \cdot S_0 = \ln\left(\frac{2}{3}\right) \approx -0.405$$

$$S_0 = \frac{\ln(2/3)}{k}$$

Because S_0 is a function of k , both steepness values produce curves that pass through exactly 6.0 at $S_{final} = 0$. The 6.0 baseline is guaranteed regardless of which side of zero the input falls on.

3. Calibrating the Steepness Parameters

The Positive Side ($k = 0.85$)

Above the 6.0 baseline, k governs how quickly an excellent performance climbs toward a perfect 10.0.

If k is set too high (e.g., $k = 1.5$), the curve is too steep: a standard great game at $+2.5\sigma$ would immediately hit 10.0, making the perfect rating far too common and eliminating the distinction between very good and genuinely exceptional. If k is too low (e.g., $k = 0.4$), the curve is too flat: a historic, multi-goal performance at $+4.0\sigma$ would struggle to surpass 8.5, unfairly punishing genuine brilliance.

$k = 0.85$ is the calibrated equilibrium. Under this slope, a strongly anomalous $+3\sigma$ performance organically approaches 9.5. Reaching a true 10.0 requires pushing beyond the normal limits of the statistical distribution — possible, but demanding. The corresponding translation parameter is:

$$S_0 = \frac{\ln(2/3)}{0.85} \approx -0.477$$

The Negative Side ($k = 0.45$)

Below the 6.0 baseline, a symmetric $k = 0.85$ would map a -2.5σ performance to approximately 3.3 — a rating psychologically reserved for catastrophic collapses: goalkeepers conceding five, defenders scoring own goals, strikers invisible for 90 minutes. A player who simply had a poor but unremarkable statistical match should settle in the 4.5 to 5.5 range, not plunge toward the floor.

$k = 0.45$ produces this gentler descent. Under this slope, a -2.5σ performance maps to approximately 4.3 and a -3σ performance to approximately 3.9. The floor remains reachable for genuinely extreme negative output, but ordinary bad games are treated proportionately. The corresponding translation parameter is:

$$S_0 = \frac{\ln(2/3)}{0.45} \approx -0.900$$

4. Algorithmic Implementation

The Python implementation explicitly calculates S_0 at runtime from k using `np.log`, preventing floating-point rounding errors that could shift the 6.0 baseline over thousands of iterations. The asymmetric branch is resolved first, so every evaluation uses a self-consistent (k, S_0) pair:

```
def _apply_sigmoid_transformation(self, raw_score: float) -> float:
    k = 0.85 if raw_score >= 0 else 0.45
    s_0: float = np.log(2 / 3) / k
    return 10 * (1 / (1 + np.exp(-k * (raw_score - s_0))))
```

The result is a 6.0-anchored curve that climbs steeply through the positive range while descending gently through the negative — matching the psychological distribution of football ratings as understood by fans, managers, and analysts alike.

VI. Macro-Context, Hybridisation & Goalkeepers

While the logistic curve successfully synthesises individual telemetry into a recognisable 1-to-10 rating, evaluating a player purely in a statistical vacuum is inherently flawed. Football is not a static environment. The match's overall momentum deeply influences a player's individual metrics; their tactical role can change depending on the phase of play, and specific positions operate under entirely different physical and data rules.

To finalise the match rating, the Analytics Engine applies three macro-resolutions. It introduces a team-level environment scalar, deploys a multi-positional hybridisation formula for players who shift roles mid-match, and completely isolates Goalkeepers into their own bespoke analytical pipeline.

The Match Supremacy Scalar

In football, it is significantly easier to string together 50 passes and register an assist when your team has 70% possession and is pinning the opponent in their own box. Conversely, a centre-back on a team facing an

absolute onslaught will naturally accumulate massive tackling and clearance numbers simply because the ball is always near them.

Statistical research into football telemetry has proven that evaluating a player in a vacuum is flawed. In fact, roughly 26% of a player's statistical variance is dictated entirely by how dominant their team was on the day^[4]. If an algorithm ignores this, it will artificially inflate the ratings of players on dominant teams (stat-padding) while systematically punishing players on struggling teams.

To neutralise this environmental bias, the Analytics Engine applies a final macro-adjustment to the 1-to-10 score: **The Match Supremacy Scalar**.

The engine calculates which team genuinely dominated the match by comparing the two teams' Expected Goals (xG). Based on this ratio, the engine applies a bidirectional adjustment:

- **The Inflation Penalty:** If a player's team heavily dominated the match, the engine applies a slight penalty to their final rating. This strips away the "easy game" padding and ensures their 9.0+ rating was actually earned through individual brilliance, not just team superiority. This penalty is strictly capped to protect elite performers from unjust punishment.
- **The Siege Bonus:** If a player's team was heavily outplayed, the engine applies a rating boost. This rewards players who maintained a solid baseline output despite being tactically besieged, recognising their performance against overwhelming odds. This bonus is likewise bounded — there is a ceiling on how much any match environment can inflate a rating, ensuring the final score always reflects the player's individual contribution.

Deep Dive: The Match Supremacy Scalar >

06a - The Match Supremacy Scalar

1. The Hierarchical Variance Problem

The architectural necessity of the Match Supremacy Scalar is derived from the statistical properties of invasion-territorial sports. Football telemetry is inherently hierarchical; a player's individual statistical output is deeply nested within the macro-performance of their team.

Recent peer-reviewed statistical analyses utilising Generalised Linear Mixed Models (GLMMs) to deconstruct commercial rating systems revealed an Intraclass Correlation Coefficient (ICC) of approximately 0.26. This indicates that 26% of the variance in professional football ratings is entirely due to macro-level hierarchical contexts, rather than the isolated actions of the player. Without a structural modifier, the algorithm would fail to separate individual brilliance from team dominance. The Match Supremacy Scalar is designed to explicitly neutralise this covariance, realigning the final output with true individual merit.

2. Expected Goals Delta and Laplace Smoothing

To quantify hierarchical team dominance, the Analytics Engine calculates the Expected Goals Ratio (x_G) between the player's team and the opponent.

However, raw division introduces a critical mathematical vulnerability: if a team completely stifles the opposition, resulting in 0.0 opponent xG, division by zero collapses the algorithm. To resolve this, and to dampen the volatility of low-event matches (e.g., 0.2 xG vs 0.1 xG), the engine applies Laplace Smoothing, adding a +1.0 constant to both numerator and denominator:

$$x_G = \frac{\text{Team xG} + 1}{\text{Opponent xG} + 1}$$

3. Logarithmic Dampening

Because team dominance operates on a curve of diminishing returns — the tactical difference between 1.0 and 2.0 xG is massive, whereas the difference between 4.0 and 5.0 xG is negligible — the engine passes the smoothed ratio through a natural logarithm ($\ln$).

The logarithm is scaled by a dampening parameter ($= 0.2$) to compress the output into a tight numerical boundary suitable for the 1-to-10 rating scale:

$$\text{Scalar} = 0.2 \times \ln(x_G)$$

4. Application and Asymmetric Bounding

The resulting scalar is mathematically **subtracted** from the player's post-sigmoid rating, creating a bidirectional contextual adjustment:

- **The Inflation Penalty** ($x_G > 1$): If a player's team dominates, the natural log evaluates to a positive number. Subtracting a positive acts as a penalty, stripping away artificial inflation from team superiority.
- **The Siege Bonus** ($x_G < 1$): If a player's team is outplayed, the natural log evaluates to a negative number. Subtracting a negative produces a positive adjustment, actively boosting the rating to reward performance under pressure.

Both directions are bounded. The scalar is clamped symmetrically after computation:

$$\text{Scalar}_{final} = \max(\min(\text{Scalar}, 0.25), -0.35)$$

The positive cap at 0.25 ensures the Inflation Penalty cannot deduct more than a quarter of a point, protecting elite performers from unjust punishment for their team's dominance. The negative floor at -0.35 ensures the Siege Bonus cannot inflate a rating by more than 0.35 points, preventing extreme possession imbalances — a team facing 5.0+ opponent xG — from compounding an already strong individual performance into an artificially elite rating. The physical interpretation: once opponent xG reaches approximately 4.3, the logarithm saturates the floor, and no further boost accumulates regardless of how much the team was dominated.

Multi-Positional Hybridisation

Modern football is increasingly positionless. Tactical fluidity often dictates that players change roles mid-match — a fullback may invert into central midfield, or a winger may transition into a second striker, depending on the game state.

However, evaluating these hybrid performances introduces a major telemetry constraint. When a user enters multiple positions for a player (e.g., "CB, RB" in the UI), the Analytics Engine only has aggregated match stats. The data is "blind" — the engine does not know if the player spent 80 minutes at CB and 10 minutes at RB, nor does it know which tackles occurred in which role.

To resolve this, the engine embraces parallel evaluation. It uses the player's entire unpartitioned match data and runs it through the independent weight matrices for each entered position, generating multiple distinct match ratings.

Collapsing these multiple ratings into a single final score presents an architectural challenge:

- **The Accountability Void (Maximum):** The engine cannot simply award the highest rating and discard the rest. If a player puts in a world-class performance in their main role but performs disastrously when asked

to cover a secondary role for a portion of the match, they must be held accountable for that poor stint. Ignoring the lower rating entirely grants them an unearned free pass.

- **The Versatility Punishment (Average):** Conversely, the engine cannot use a simple average. If a player earns a 9.0 in their primary role but a 6.0 in the secondary role due to unfamiliarity or lack of volume, a pure average (7.5) unjustly destroys their rating, punishing them far too heavily for stepping in to fill a tactical gap.

To bridge this gap, the engine deploys a proprietary **Alpha Drag** system. The algorithm identifies the player's highest calculated rating as their primary role, anchors the final score there, and applies a fractional drag toward the mean based on the lower scores. Crucially, the size of this drag is not fixed — it is dynamically calibrated by how tactically similar the positions are to one another. A player who blends two closely related roles receives more drag because the secondary rating is a meaningful signal; a player who bridges two tactically orthogonal roles receives far less drag because the secondary rating is an artefact of the position mismatch rather than genuine poor performance.

Deep Dive: The Alpha Drag Coefficient >

06b - Multi-Position Alpha Drag

1. The Telemetry Constraint (Blind Aggregation)

When a player plays multiple tactical roles in a single match, the engine receives an unpartitioned telemetry array (T_{match}). Because the system lacks temporal parsing (minutes played per position) and spatial parsing (event locations), it cannot isolate which specific statistics belong to which tactical role.

To bypass this blind aggregation, the algorithm executes Parallel Matrix Evaluation. It processes T_{match} through n distinct Positional Weight Vectors corresponding to the user's input, generating a set of positional ratings $R = r_1, r_2, \dots, r_n$.

2. Pre-Calculation: Mirror Pair Deduplication

Before the hybrid formula runs, the engine collapses lateral mirror pairs from the position list. The four bilateral pairs — LB/RB, LWB/RWB, LM/RM, and LW/RW — describe the same tactical role on opposite flanks. When both sides of a pair appear in the input, the lower-rated entry is dropped and only the higher-rated is retained.

This step prevents bilateral listings from artificially inflating the position count or distorting the drag calculation. A user who enters "CB, RB, LB" intends to describe a centre-back who covered both fullback positions, not a three-way hybrid; after deduplication the engine treats it as "CB, RB" (or whichever fullback side scored higher), which is the architecturally correct interpretation.

If the deduplication leaves only a single position, the hybrid formula is skipped entirely and that rating is returned directly as the final score.

3. The Alpha Drag Formula

To synthesise the deduplicated set R into a single scalar without falling victim to maximum-value loopholes or punitive averaging, the engine identifies the maximum rating (R_{max}), the arithmetic mean (R_{mean}), and the minimum (R_{min}) across all retained positions.

The final hybrid score is:

$$R_{\text{hybrid}} = R_{\text{max}} - \underbrace{\alpha, (R_{\text{max}} - R_{\text{mean}})}_{\text{drag}} + \underbrace{\beta \times \max(0, R_{\text{min}} - \tau)}_{\text{versatility bonus}}$$

Drag term: α is the dynamic drag coefficient (derived in Section 4). It pulls the final score from R_{max} toward R_{mean} , holding the player partially accountable for weaker performances in secondary roles without collapsing to a punitive average.

Versatility bonus: $\beta = 0.15$ and $\tau = 6.5$. If the player's *worst* positional rating exceeds the threshold, a small additive bonus fires proportional to the surplus above it. A player who is genuinely above average in every listed role demonstrates authentic positional intelligence — the engine rewards this rather than neutralising it through drag alone. The threshold is set above the 6.0 psychological baseline by half a point, so the bonus only activates when the player is credibly above average even at their weakest position.

4. The Dynamic Alpha Coefficient

Rather than hardcoding a fixed drag weight, the engine calibrates α dynamically based on the tactical relationship between the positions being hybridised.

Positional similarity is measured using **cosine similarity** between the z-scored historical stat mean profiles of the highest-rated position and each secondary position. Each position is represented by its mean stat values from the offline calibration data, centred and scaled against the cross-position distribution — capturing how each role's typical statistical footprint *deviates from the positional average* rather than reflecting the raw magnitudes. This approach is independent of the weighting choices made during PCA and focuses purely on the characteristic statistical differences between tactical roles.

$$\cos(\theta) = \frac{\mathbf{z}_1 \cdot \mathbf{z}_2}{|\mathbf{z}_1|, |\mathbf{z}_2|}$$

where $\mathbf{z}_p$ is the z-scored stat mean profile for position p . Negative cosine values — produced by anti-correlated roles such as CB and ST — are clamped to zero: orthogonal or opposite positions generate no drag at all, leaving the hybrid rating at R_{max} .

The mean cosine similarity across all secondary positions scales the base drag coefficient:

$$\alpha = 0.50 \times \text{c}\ddot{\text{o}}\text{s}(\theta)$$

```
max_idx: int = int(np.argmax(calculated_ratings))
max_pos: str = positions_played[max_idx]
other_positions: list[str] = [
    p for i, p in enumerate(positions_played) if i != max_idx
]

mean_similarity: float = float(np.mean([
    self._positional_cosine_similarity(max_pos, p)
    for p in other_positions
]))
alpha: float = 0.50 * mean_similarity # ALPHA_BASE = 0.50

r_max: float = max(calculated_ratings)
r_mean: float = float(np.mean(calculated_ratings))
r_min: float = min(calculated_ratings)

drag: float = alpha * (r_max - r_mean)
```

```
bonus: float = 0.15 * max(0.0, r_min - 6.5)
hybrid_rating: float = r_max - drag + bonus
```

Why this matters — two contrasting cases:

Case A — Tactically Related Positions (e.g., CB and RB): Both stat profiles are characterised by high defensive output and modest attacking figures relative to the cross-position average. The profiles are closely aligned, producing a high cosine similarity (e.g., ≈ 0.80). Alpha is therefore $0.50 \times 0.80 \approx 0.40$. The secondary RB rating exerts meaningful drag because it is a legitimate signal — the player's aggregate stats genuinely reflect how well they fulfilled the duties of both roles.

Case B — Tactically Orthogonal Positions (e.g., CB and ST): The CB profile deviates from the cross-position average in the direction of high tackles and possession won but low goals and shots; the ST profile deviates in the opposite direction. The profiles are anti-correlated, producing a cosine value near zero (clamped to 0.0). Alpha collapses to $0.50 \times 0.00 = 0.00$. The secondary ST rating contributes nothing to the hybrid. This is correct — the unpartitioned CB statistics are guaranteed to generate a poor ST rating not because the player performed badly as a striker, but because the aggregate telemetry was never designed to be evaluated through a striker's weight matrix. Allowing that artefact rating to drag the true CB performance would be a mathematical injustice.

The dynamic calibration therefore ensures that the Alpha Drag system rewards genuine positional intelligence — correctly penalising a player who was tactically responsible for a closely related secondary role while protecting players from phantom drag generated by an orthogonal position assignment they spent minimal time fulfilling.

Goalkeeper Isolation Heuristics

The structural realities of the goalkeeper position render the standard analytics algorithm entirely obsolete. This is not merely a tactical difference; it is a fundamental data constraint. For outfield players, the Analytics Engine receives a 17-metric telemetry matrix measuring progression, passing, distance, and threat. For goalkeepers, the engine receives a completely different, highly specialised 11-metric matrix consisting purely of reactive events — shots against, saves, punch saves, rush saves, penalty metrics, and shootout data.

Attempting to force an 11-metric reactive array through a 17-dimensional PCA weight matrix designed for outfielders would cause the linear algebra to collapse mathematically. Therefore, the engine completely isolates goalkeepers from the outfield architecture, routing them into a bespoke dedicated pipeline that also bypasses the Bayesian smoothing and Z-score standardisation steps used for outfield players.

The core of this pipeline is **Expected Goals Prevented (xGP)**. Rather than simply counting raw saves, the engine calculates the opponent's total Expected Goals and subtracts the goals actually conceded. This creates a context-aware measurement of shot-stopping value that distinguishes a goalkeeper who faced 0.3 xG and made no saves from one who faced 3.0 xG and made ten.

Furthermore, the goalkeeper pipeline deliberately reverses several contextual heuristics used for outfielders to reflect their reactive role. The most prominent example is the **Inverted Clean Sheet Bonus**. When a defender keeps a clean sheet against low xG, they receive a massive boost — they actively smothered the opposition. When a goalkeeper keeps a clean sheet against low xG, the boost is minimal — they were largely a passenger in a game dominated by their own defence. Conversely, if the defence collapses and allows 2.5 xG but the goalkeeper still secures a clean sheet, they trigger a "Bailout" bonus and receive a maximum spike for single-handedly carrying the team.

Beyond the clean sheet logic, the pipeline applies multiple layers of contextual adjustment to the raw heuristic score before and after the sigmoid, handling edge cases from a completely quiet game to a penalty shootout.

The full pipeline architecture is in the deep dive below.

Deep Dive: Goalkeeper Heuristics >

06c - Goalkeeper Heuristics

1. Matrix Dimensionality Mismatch

Outfield ratings are computed as a 17-element linear combination (dot product) of standardised Z-scores, mapped against a 17×9 PCA weight matrix. Goalkeeper telemetry is fundamentally incompatible with this architecture, as the engine receives a structurally distinct 11-element array restricted entirely to reactive metrics (`shots_against` , `shots_on_target` , `saves` , `goals_conceded` , `save_success_rate` , `punch_saves` , `rush_saves` , `penalty_saves` , `penalty_goals_conceded` , `shoot_out_saves` , `shoot_out_goals_conceded`).

To resolve this dimensionality mismatch, the Analytics Engine routes goalkeepers to a dedicated `calculate_gk_rating` method. This method bypasses the Bayesian smoothing and Z-score standardisation pipeline used for outfield players entirely — only a small number of volume metrics receive temporal scaling — and instead derives a standalone mathematical heuristic (H_{GK}), standardises it against historical goalkeeper distributions, and passes the result through a multi-layer adjustment pipeline.

2. The Core Heuristic Formulation

The base performance of a goalkeeper is captured via a custom synthetic formula, heavily weighted toward Expected Goals Prevented (xGP), where $xGP = \text{Opponent xG} - \text{Goals Conceded}$:

$$H_{GK} = (xGP \times 1.5) + \left(\ln(\text{Saves} + 1) \times \frac{\text{Save\%}}{100} \right)$$

Mathematical justifications:

- **xGP Weighting (1.5):** Anchoring the formula to xGP prioritises shot quality over shot volume. A goalkeeper who faces 3.0 xG and concedes one goal prevented 2.0 expected goals; one who faces 3.0 xG and concedes three prevented nothing. The 1.5 multiplier ensures this distinction dominates the heuristic.
- **Logarithmic Volume Dampening:** Passing the raw save count through $\ln(\text{Saves} + 1)$ dampens the impact of facing a high volume of low-quality shots. Facing 12 routine long-range efforts is not equivalent to facing 12 dangerous close-range attempts; the logarithm prevents shot volume alone from inflating the score.
- **Efficiency Scaling:** The logarithmic volume is multiplied by the save percentage, heavily penalising goalkeepers who make numerous saves but still concede highly preventable goals.

Once H_{GK} is calculated, it is standardised into a Z-score using pre-calculated historical goalkeeper mean and standard deviation:

$$S_{raw} = \frac{H_{GK} - \mu_{GK}}{\sigma_{GK}}$$

3. Pre-Sigmoid Score Stabilisation

The heuristic formula contains a save term — $\ln(\text{Saves} + 1) \times \text{Save\%}/100$ — that is structurally unreliable at low shot volumes. A goalkeeper who faces one routine shot and saves it generates a disproportionately high save contribution that does not reflect a genuinely demanding performance.

Rather than patching this with a series of conditional floor overrides, the engine stabilises the raw score through two sequential steps before any additive bonuses are applied.

Step 1 — Volume Confidence Shrinkage

At low shot volumes the save term cannot be trusted, so the engine blends the full heuristic Z-score toward a reduced signal that contains only the xGP component — the one part of the formula that remains meaningful regardless of shot count. This anchor is:

$$\text{anchor} = \frac{xGP \times 1.5}{\sigma_{GK}}$$

The blend is governed by a continuous shrink factor derived from shot volume:

$$\text{shrink} = \min \left(1.0, \sqrt{\frac{\text{shots_against}}{K}} \right), \quad K = 5.0$$

The stabilised score is:

$$S_{raw} \leftarrow \text{anchor} \times (1 - \text{shrink}) + S_{raw} \times \text{shrink}$$

At zero shots the shrink factor is 0 and S_{raw} collapses entirely to the anchor — a goalkeeper who faced nothing is evaluated purely on the gap between expected and actual concessions. At $K = 5.0$ normalised shots the factor reaches 1.0 and the full heuristic score passes through unchanged. In between the two signals blend continuously: a goalkeeper who faced three shots contributes approximately $\sqrt{3/5} \approx 0.77$ of their heuristic score and 0.23 of the xGP anchor.

Crucially, the anchor is non-negative for any positive xGP, so a goalkeeper who prevented goals in a quiet match is not penalised by this step — the unreliable save term is simply replaced with a signal that can be trusted at low volumes.

```
shrink = min(1.0, np.sqrt(shots_against / self.GK_SHOTS_CONFIDENCE_K)) # K = 5.0
xgp_anchor = (xgp * 1.5) / gk_std if gk_std > 0 else 0.0
raw_score = xgp_anchor * (1 - shrink) + raw_score * shrink
```

Step 2 — Absolute Final Floor

An unconditional floor at -1.25 is applied after shrinkage, regardless of circumstances. No goalkeeper can enter the additive bonus stage with a raw score below this value. This is the mathematical backstop — it ensures that even genuinely catastrophic performances produce a minimum post-sigmoid rating rather than a potentially undefined edge case.

```
raw_score = max(raw_score, -1.25)
```

4. Pre-Sigmoid Bonus Adjustments

After the stabilisation steps, two sets of additive bonuses are applied to S_{raw} before the sigmoid runs.

Additive Performance Bonuses

- **Penalty Heroics:** $+0.50$ per penalty save and $+0.50$ per shootout save. These are combined into a single operation (`total_penalty_saves = penalty_saves + shoot_out_saves`). Each save

represents an absolute, match-defining moment of individual excellence that the heuristic formula cannot adequately capture.

- **Reliable Shift:** If `goals_conceded == 1`, `xgp > 0`, and `save_success_rate >= 80%`, the keeper receives $+0.25$. Conceding one goal while saving more xG than was conceded and maintaining a high save rate is a quietly excellent performance that deserves recognition above the narrow-miss floor.

The Inverted Clean Sheet Bonus

A clean sheet is registered only when `goals_conceded == 0`, `penalty_goals_conceded == 0`, and `shoot_out_goals_conceded == 0`. When this condition is met, a further bonus is applied to S_{raw} . While defenders are rewarded most for clean sheets against *low* xG (indicating defensive control), goalkeepers are rewarded most for clean sheets against *high* xG (indicating a goaltending performance that single-handedly preserved the result):

```
# 1. Passenger: Low threat, keeper had little to do
if xg_against <= 1.0 and xgp < 0.95:
    return raw_score + 0.30

# 2. Bailout: High threat, keeper made multiple crucial saves
elif xg_against > 2.0 and saves >= 5:
    return raw_score + 0.80

# 3. Standard: Any other clean sheet scenario
else:
    return raw_score + 0.50
```

Once all pre-sigmoid adjustments are complete, the sigmoid transformation is applied to the final S_{raw} . The Match Supremacy Scalar is then subtracted from the post-sigmoid rating — identical in application to the outfield pipeline — and the result is clamped to $[0.0, 10.0]$.

VII. Conclusion

The tension between statistical objectivity and tactical reality has historically hampered the quantification of football performance. Commercial rating systems and native in-game algorithms often rely on rigid, one-size-fits-all linear models. These models fail to account for asymmetric responsibilities across positions, aggressively punish players for executing specific managerial instructions, and unquestioningly reward players for hierarchical dominance within their team.

The custom Analytics Engine detailed in this whitepaper was engineered to resolve these epistemological flaws from the ground up. Before any scoring logic runs, the pre-processing pipeline (Section II) neutralises EA FC's variable half-length clock through temporal standardisation, then applies a three-tier Bayesian smoothing architecture — calibrated separately for high-frequency, medium-frequency, and rare-event metrics — to protect the algorithm against the substitute volatility that plagues simpler implementations. In Section III, the engine synthesises a spatial threat proxy from physical and technical telemetry, compresses right-skewed statistics through logarithmic normalisation, and applies continuous confidence-weighted volume masking to ensure efficiency metrics are only trusted once meaningful sample sizes are accumulated.

The offline weight derivation (Section IV) then grounds the algorithm in data rather than intuition. By running a possession-adjusted historical dataset through a developer-guided Principal Component Analysis — where human tactical expertise steers the variance extraction while the Minimum Covariance Determinant filters out simulation anomalies — the engine derives position-specific weight vectors that accurately reflect the true

statistical DNA of each role without arbitrarily hand-tuning a single number. Most importantly, the engine wraps this linear algebra in rigorous, context-aware heuristics (Sections V & VI). A tactical build-up isolation multiplier prevents ghost performances from receiving unearned goal bonuses; a square-root impact scalar naturally compresses abbreviated appearances toward the baseline; an asymmetric sigmoid provides a steeper climb above 6.0 than the descent below it; a dynamic Alpha Drag coefficient — calibrated by the cosine similarity between positional stat profiles — hybridises multi-role performances without phantom drag from orthogonal assignments; and the GLMM-backed Match Supremacy Scalar, bounded symmetrically in both directions, strips away team-dominance inflation and rewards individual quality under pressure.

The resulting output is a robust, dynamic 0.0 to 10.0 rating system that does not merely count what a player did, but mathematically evaluates *how* and *why* they did it — within the specific tactical context of their position, their minutes played, and the match environment around them. It successfully mimics the subjective, nuanced eye of an elite football analyst while maintaining the strict, reproducible objectivity of a data science framework.

Appendix: Complete Positional Heuristics Reference >

Appendix A - Positional Heuristics

This appendix is the complete technical reference for all outfield positional modifiers as implemented in `match_ratings_service.py`. Values are sourced directly from the live code; no values are estimated or approximated.

Universal Mechanisms

Bottleneck Synergy Every scaling bonus in the engine uses a bottleneck architecture. A bonus activates only when a player simultaneously exceeds a threshold across two paired metrics, and the scaling rate is driven by the weaker of the two:

$$\text{bonus} = R \times (\min(Z_A, Z_B) - T) \quad \text{when } \min(Z_A, Z_B) > T$$

A player who is elite in one metric but average in its pair receives no bonus until both are above the threshold. This prevents a single extraordinary stat from generating a reward while masking a deficiency in the complementary skill.

Goal and Assist Multipliers Goal bonuses are efficiency-adjusted rather than flat. The engine computes an *above-expected* component using a crude 0.20 xG-per-shot estimate, with a per-goal floor ensuring scoring always contributes positively regardless of shot volume:

$$\begin{aligned} \text{above_expected} &= \max(\text{goals} - \text{shots} \times 0.20, \text{goals} \times 0.40) \\ \text{goal_contribution} &= \text{above_expected} \times G_{pos} \end{aligned}$$

Assist bonuses remain a flat multiplier: $\text{assist_contribution} = \text{assists} \times A_{pos}$.

The **tactical isolation multiplier** derived in Phase III (Section V) is applied to goal and assist bonuses for all midfield and attacking positions (CDM, CM, CAM, WM, Winger, ST). If the player's average build-up Z-score falls below -1.0 , this multiplier decays exponentially between 0 and 1, reducing bonuses for players statistically absent from the build-up phase. Defensive positions (CB, Fullback, Wingback) are not subject to the isolation multiplier.

Efficiency Metrics Percentage-based metrics (pass accuracy, shot accuracy, dribble success rate, tackle success rate) are regulated by the continuous logistic volume masking system described in Section III. They are not subject to universal hard floors in the positional modifier layer; only position-specific floors explicitly coded in the modifier are listed below.

1. Attackers

Striker (ST)

Z-Score Floors

Metric	Floor
tackles_p90_z	−0.50
tackle_success_rate_z	−0.50
possession_won_p90_z	−0.50
goals_p90_z	−2.00 — Ghosting Forgiveness
assists_p90_z	−2.00 — Ghosting Forgiveness
non_goal_shots_p90_z	−2.00 — Ghosting Forgiveness
offsides_p90_z	−1.50 — Offside Forgiveness

Output Multipliers Goal coefficient $G_{pos} = 1.5$, Assists $\times 1.1$ (both scaled by isolation multiplier).

Complete Forward Bonus Activates when $\min(\text{passes_z}, \text{dribbles_z}) > 1.5$; scales at $+0.25$ per σ above threshold.

Wasteful Finisher Penalty Assigns $0.20 \times \text{G}$ per shot. If shots > 3 and finishing deficit > 0.75 : penalty = deficit $\times 0.25$, capped at -0.80 .

Black Hole Penalty If possession lost > 4 and turnover ratio > 1.5 : penalty = excess losses $\times 0.08$, capped at -0.60 .

Positive involvements = passes + dribbles + shots. Excess losses = possession lost – positive involvements.

Target Man Bonus If positive involvements ≥ 15 and retention ratio > 4.0 : bonus = excess retention $\times 0.02$, capped at $+0.40$.

Excess retention = positive involvements – (possession lost $\times 3.0$).

Winger (RW / LW)

Z-Score Floors

Metric	Floor
tackles_p90_z	−0.50 — Defensive Absolution
tackle_success_rate_z	−0.50 — Defensive Absolution
possession_won_p90_z	−0.50 — Defensive Absolution
fouls_committed_p90_z	−1.50
possession_lost_p90_z	−1.50
offsides_p90_z	−2.00 — Offside Forgiveness

Output Multipliers Goal coefficient $G_{pos} = 1.3$, Assists $\times 1.0$ (both scaled by isolation multiplier).

Direct Threat Bonus Activates when $\min(\text{dribbles_z}, \text{xt_z}) > 1.5$; scales at $+0.25$ per σ above threshold.

Wide Playmaker Bonus Activates when $\min(\text{passes_z}, \text{xt_z}) > 1.5$; scales at $+0.20$ per σ above threshold.

High Press Bonus Activates when $\min(\text{tackles_z}, \text{poss_won_z}) > 1.0$; scales at $+0.15$ per σ above threshold.

Wastefulness Penalty If $\text{shots} \geq 3$ and $\text{goals} = 0$: $\text{penalty} = (\text{shots} - 2) \times 0.10$.

2. Midfielders

Central Attacking Midfielder (CAM)

Z-Score Floors

Category	Metrics	Floor
Defensive	tackles_p90_z , tackle_success_rate_z , possession_won_p90_z	−0.50
Passenger	passes_p90_z , dribbles_p90_z , non_goal_shots_p90_z , distance_covered_p90_z , distance_sprinted_p90_z , xt_bonus_p90_z	−1.00
Detriment	fouls_committed_p90_z , possession_lost_p90_z , offsides_p90_z	−1.50

Output Multipliers Goal coefficient $G_{pos} = 0.9$, Assists $\times 0.75$ (both scaled by isolation multiplier).

Maestro Bonus Activates when $\min(\text{passes_z}, \text{xt_z}) > 1.5$; scales at $+0.25$ per σ above threshold.

Shadow Striker Bonus Activates when $\min(\text{non_goal_shots_z}, \text{xt_z}) > 1.5$; scales at $+0.20$ per σ above threshold.

Modern 10 Bonus Activates when $\min(\text{tackles_z}, \text{poss_won_z}) > 1.0$; scales at $+0.25$ per σ above threshold.

Central Midfielder (CM)

Z-Score Floors None applied in the positional modifier.

Output Multipliers Goal coefficient $G_{pos} = 1.0$, Assists $\times 0.75$ (both scaled by isolation multiplier).

Enforcer Bonus Activates when $\min(\text{tackles_z}, \text{poss_won_z}) > 1.5$; scales at $+0.25$ per σ above threshold.

Progression Engine Bonus Activates when $\min(\text{passes_z}, \text{dribbles_z}) > 1.2$; scales at $+0.25$ per σ above threshold.

Clean Sheet Bonus Flat $+0.15$ if opponent goals = 0 and minutes ≥ 60 .

Central Defensive Midfielder (CDM)

Z-Score Floors

Metric	Floor
fouls_committed_p90_z	−1.00 — Dark Arts Leniency

Output Multipliers Goal coefficient $G_{pos} = 0.6$, Assists $\times 0.45$ (both scaled by isolation multiplier).

Destroyer Bonus Activates when $\min(\text{tackles}_z, \text{poss_won}_z) > 1.5$; scales at $+0.25$ per σ above threshold.

Deep-Lying Playmaker Bonus Activates when $\min(\text{passes}_z, \text{dribbles}_z) > 1.5$; scales at $+0.25$ per σ above threshold.

Reliable Pivot (*requires minutes ≥ 60 , pass accuracy $\geq 92\%$, and passes_z > 1.0*)

Condition	Bonus
Possession lost = 0	+0.35 — Perfect Metronome
Possession lost ≤ 1	+0.20 — Reliable Shift

Clean Sheet Bonus Flat $+0.20$ if opponent goals = 0 and minutes ≥ 60 .

Wide Midfielder (RM / LM)

Z-Score Floors

Metric	Floor
fouls_committed_p90_z	-1.50
possession_lost_p90_z	-1.50
offsides_p90_z	-1.50

Output Multipliers Goal coefficient $G_{pos} = 0.75$, Assists $\times 0.55$ (both scaled by isolation multiplier).

Two-Way Engine Bonus Activates when $\min(\text{passes}_z, \text{tackles}_z) > 1.0$; scales at $+0.25$ per σ above threshold.

Wide Progressor Bonus Activates when $\min(\text{xt}_z, \text{dribbles}_z) > 1.0$; scales at $+0.20$ per σ above threshold.

Clean Sheet Bonus Flat $+0.15$ if opponent goals = 0 and minutes ≥ 60 .

3. Defenders

Centre Back (CB)

Z-Score Floors None applied in the positional modifier.

Output Multipliers Goal coefficient $G_{pos} = 0.75$, Assists $\times 0.55$.

Dominant Stopper Bonus Activates when $\min(\text{tackles}_z, \text{poss_won}_z) > 1.5$; scales at $+0.25$ per σ above threshold.

Ball-Playing Defender Bonus Activates when $\min(\text{passes}_z, \text{poss_won}_z) > 1.0$; scales at $+0.20$ per σ above threshold.

Dynamic Clean Sheet Bonus

Opponent xG	Bonus
≤ 1.0	+0.50 — Defensive Dominance
≥ 2.0	+0.15 — Fortunate Clean Sheet
Otherwise	+0.35 — Standard

Collapse Penalty Flat -0.30 if opponent goals ≥ 3 and minutes ≥ 60 .

Fullback (LB / RB)

Conditional Z-Score Floors The Tactical Instruction Floor is conditional on whether the Third CB archetype is active. The archetype is active when $\min(\text{tackles}_z, \text{poss_won}_z) > 1.0$.

Metric	Third CB Active	Third CB Inactive
dribbles_p90_z	0.00 (lifted to neutral)	-0.50
xt_bonus_p90_z	0.00 (lifted to neutral)	-0.50

Output Multipliers Goal coefficient $G_{pos} = 0.5$, Assists $\times 0.4$.

Third CB Bonus Activates when $\min(\text{tackles}_z, \text{poss_won}_z) > 1.0$; scales at $+0.25$ per σ above threshold.

Express Train Bonus Activates when $\min(\text{sprint}_z, \text{xt}_z) > 1.0$; scales at $+0.20$ per σ above threshold.

Wide Playmaker Bonus Activates when $\min(\text{passes}_z, \text{dribbles}_z) > 1.0$; scales at $+0.15$ per σ above threshold.

Dynamic Clean Sheet Bonus Identical to CB logic.

Collapse Penalty Flat -0.30 if opponent goals ≥ 3 and minutes ≥ 60 .

Wingback (LWB / RWB)

Z-Score Floors None applied in the positional modifier.

Output Multipliers Goal coefficient $G_{pos} = 0.75$, Assists $\times 0.55$.

Relentless Engine Bonus Activates when $\min(\text{sprint}_z, \text{xt}_z) > 1.5$; scales at $+0.20$ per σ above threshold.

Two-Way Flank Bonus Activates when $\min(\text{tackles}_z, \text{poss_won}_z) > 1.0$; scales at $+0.25$ per σ above threshold.

Dynamic Clean Sheet Bonus Identical to CB logic. No collapse penalty.

Footnotes

Footnotes

1. Drs Raffaele Poli, Loïc Ravenel and Roger Besson (2021). Monthly Report 68: Analysis of distances covered in professional football. *CIES Football Observatory* [Available Here](#) ↩
2. Kubayi A. (2019). Evaluation of match-running distances covered by soccer players during the UEFA EURO 2016. *South African journal of sports medicine*, 31(1), v31i1a6127. [Available Here](#) ↩
3. Matthew Herring-Rogers. Distance in football matters: Use it wisely. *PlayerData*. [Available Here](#) ↩
4. James Ball, Minh Huynh and Matthew C. Varley (2025). Comparing player rating systems as a metric for assessing individual performance in soccer. *Journal of Sports Sciences* [Available Here](#) ↩